

BCSE498J- Project - II

UNIFIED OPTIMIZATION PIPELINE: INTEGRATING NAS, PRUNING, AND QUANTIZATION FOR EDGE DEVICES

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BCB0198 MANYA SINGHAL

22BCE2560 AYUSHI JHA

22BCE2119 BOMBAYWALA MANUSHI SWAPNIL

Under the Supervision of

Prof. Swetha N G

Assistant Professor Sr. Grade 1

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

Digitally signed by VIT University.
Timestamp: 2026-04-27 14:22:26 UTC.

DECLARATION

We hereby declare that the project entitled “UNIFIED OPTIMIZATION PIPELINE: INTEGRATING NAS, PRUNING, AND QUANTIZATION FOR EDGE DEVICES” submitted by us, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by us under the supervision of Prof. Swetha N G, School of Computer Science and Engineering.

We further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

MANYA SINGHAL (22BCB0198)

Date :

AYUSHI JHA (22BCE2560)

BOMBAYWALA MANUSHI SWAPNIL (22BCE2119)

CERTIFICATE

This is to certify that the project entitled “UNIFIED OPTIMIZATION PIPELINE: INTEGRATING NAS, PRUNING, AND QUANTIZATION FOR EDGE DEVICES” submitted by MANYA SINGHAL (22BCB0198), AYUSHI JHA (22BCE2560), BOMBAYWALA MANUSHI SWAPNIL (22BCE2119) , School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering , is a record of bonafide work carried out by the students under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Signature of the Guide

Date :

Internal Examiner

External Examiner

Head of the Department

ACKNOWLEDGEMENT

We MANYA SINGHAL (22BCB0198), AYUSHI JHA (22BCE2560), BOMBAYWALA MANUSHI SWAPNIL (22BCE2119) are deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled us to explore and develop our ideas to their fullest potential.

Our sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

We express our profound appreciation to Dr. Mythili T, Head of the Department of Analytics and Dr. Boominathan P, Head of the Department of Software Systems, for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of our project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced our learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving our project goals.

We are immensely thankful to our project supervisor, Prof. Swetha N G, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining our ideas and methodologies. This support has not only contributed to the success of this project but has also enriched our overall academic experience.

We extend our heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

Executive Summary	v
List of Tables	vi
List of Figures	vii
List of Abbreviations	viii
List of Symbols	ix
1 INTRODUCTION	1
1.1 Background	2
1.2 Motivation	3
1.3 Scope of the project	4
2 PROJECT DESCRIPTION AND GOALS	5
2.1 Literature review	5
2.2 Research Gap	15
2.3 Objectives	16
2.4 Problem statement	16
2.5 Project plan	17
3 TECHNICAL SPECIFICATION	19
3.1 Requirements	19
3.1.1 Functional and Non-Functional Requirements	19
3.2 Feasibility Study	20
3.3 System Specification	21
4 SYSTEM DESIGN	23
4.1 System Architecture	23
4.2 Data Flow Diagram	25
5 METHODOLOGY AND TESTING	27
5.1 Methodology	27
5.1.1 Overview	27

5.1.2	Dataset Preparation	27
5.1.3	Baseline Models	28
5.1.4	Structured Pruning of Baseline Models	28
5.1.5	Neural Architecture Search: DARTS	29
5.1.6	DARTS Network Pruning	30
5.1.7	Model Evaluation and Comparison	33
5.1.8	ONNX Export and Edge Deployment	33
5.1.9	ONNX Runtime Benchmarking	33
5.2	Baseline MobileNetV2	34
5.2.1	Architecture Overview	34
5.2.2	Depthwise Separable Convolution	34
5.2.3	Inverted Residual Block with Linear Bottleneck	35
5.2.4	Network Structure	35
5.2.5	Training	36
5.3	Pruned MobileNetV2	36
5.3.1	Filter Importance and Mask Construction	36
5.3.2	Layer Selection Policy	37
5.3.3	Compact Model Reconstruction	37
5.3.4	Fine-tuning	37
5.4	Baseline MobileNetV3-Small	38
5.4.1	Architecture Overview	38
5.4.2	Hard-Swish and Squeeze-and-Excitation	39
5.4.3	InvertedResidual Block with SE	39
5.4.4	Network Structure	39
5.4.5	Training	39
5.5	Pruned MobileNetV3-Small	40
5.5.1	Filter Importance and Mask Construction	40
5.5.2	Block Reconstruction	40
5.5.3	Fine-tuning	40
5.6	Baseline EfficientNet	41
5.6.1	Architecture Overview	41
5.6.2	Swish Activation and MBConv Block	42

5.6.3	Network Structure	42
5.6.4	Training	42
5.7	Pruned EfficientNet	42
5.7.1	Block Analysis, Filter Importance, and Mask Construction . . .	43
5.7.2	MBConv Block Reconstruction	43
5.7.3	Fine-tuning	43
5.8	Testing	45
5.8.1	Overview	45
5.8.2	Test Environment	45
5.8.3	Test Cases	46
5.8.4	Evaluation Metrics	49
6	PROJECT IMPLEMENTATION	51
6.1	Development Environment	51
6.1.1	Hardware	51
6.1.2	Software Stack	51
6.1.3	Reproducibility	52
6.2	Dataset Preparation Implementation	52
6.3	Baseline Model Implementation	53
6.3.1	MobileNetV2	53
6.3.2	MobileNetV3-Small	53
6.3.3	EfficientNet	53
6.4	Structured Pruning Implementation	54
6.4.1	Filter Importance Scoring	54
6.4.2	Model Reconstruction Utilities	55
6.4.3	MobileNetV2 Pruning	56
6.4.4	MobileNetV3-Small Pruning	56
6.4.5	EfficientNet Pruning	58
6.5	DARTS Implementation	58
6.5.1	Search Phase	59
6.5.2	Discretisation and Final Network Training	60
6.5.3	DARTS Pruning	60
6.6	ONNX Export Implementation	60

6.7	Raspberry Pi Edge Deployment Setup	62
6.7.1	Hardware Configuration	62
6.7.2	Operating System and Software Configuration	63
6.7.3	Model Transfer and Validation	63
6.8	Edge Deployment Benchmarking Implementation	64
7	RESULTS AND DISCUSSION	66
7.1	Theoretical Architecture Optimization	66
7.2	Edge Hardware Inference	67
7.3	Impact of INT8 Quantization on ARM Cortex Architectures	68
7.4	Comprehensive Analysis of Pipeline Performance	69
7.4.1	Cross-Architecture Efficacy	69
7.5	Qualitative Inference Validation	70
7.5.1	Benchmarking Results Visualisation	73
8	CONCLUSION AND FURTHER ENHANCEMENTS	76
8.1	Conclusion	76
8.2	Further Enhancements	76
9	APPENDIX	84

EXECUTIVE SUMMARY

The important limitation in the physical availability of Deep Convolutional Neural Networks (CNNs) onto edge Internet of Things (IoT) hardware is the extreme limitations on computations and memory. This work presented a uniform optimization pipeline that pulls Differentiable Architecture Search (DARTS) and L1-Norm structural pruning to close the distance between abstract model compressions and practical hardware step up. This pipeline is experimentally evaluated using CIFAR-10 on three varied baseline CNNs (MobileNetV2, MobileNetV3, EfficientNet) in two distinct phases: cloud-based theoretical optimization followed by physical deployment and intensive hardware benchmarking of a resource-constrained Raspberry Pi 4 (ARM Cortex-A72 CPU).

Significant and multi-objective performance improvements were observed: over 99% of parameter compression was achieved in theory with preserved predictive confidence and tightened class boundaries for visually similar objects; when implemented on hardware, the pipeline successfully overcome the severe CPU memory limitations of purely structurally pruned models, utilizing the DARTS architecture rebuilding process for colossal latency reductions – achieving a up to 6.68x hardware speedup and sub-millisecond inference times for mobile CNN variants; and hardware constraints showed that INT8 quantization was sub-optimal for ARM CPUs with no built-in hardware integer accelerators.

Ultimately, this research proves that computationally heavy neural architectures can be mathematically rescued and optimized for real-time, low-power edge deployment without reliance on expensive, dedicated neural processing units.

LIST OF TABLES

2.1	Literature Review of DARTS-based Neural Architecture Search and Optimization Methods	6
3.1	Hardware and Software Environment Specifications	22
3.2	Comprehensive Hyperparameter Configurations	22
5.1	MobileNetV2 architecture configuration for CIFAR-10.	35
5.2	MobileNetV3-Small classifier head adapted for CIFAR-10.	39
5.3	EfficientNet classifier head adapted for CIFAR-10.	42
6.1	Software environment used for training, pruning and deployment. . . .	52
7.1	<i>Comprehensive Performance Comparison Across Baseline Architectures on Google Colab</i>	67
7.2	<i>Edge Hardware Performance on Raspberry Pi (FP32 Inference)</i>	68
7.3	FP32 vs. INT8 Inference Penalty on Raspberry Pi	69
7.4	Image Inference Metrics (Class: Truck) and Global Model Performance Across All Architectures	71

LIST OF FIGURES

4.1	System Architecture	23
4.2	Consolidated data flow for all three baseline and pruned model pipelines	26
5.1	DARTS pipeline and pruning with fine-tuning producing	32
5.2	Structured pruning pipeline for MobileNetV2.	38
5.3	Structured pruning pipeline for MobileNetV3-Small, showing the stage-by-stage process.	41
5.4	Structured pruning pipeline for EfficientNet , showing the stage-by-stage process from trained baseline.	44
6.1	Raspberry Pi 4 Model B edge deployment platform.	62
6.2	Imager configuration.	62
6.3	Raspberry Pi terminal showing the Python environment setup with ONNX Runtime 1.16 and NumPy 1.24.	63
7.1	<i>Qualitative inference validation (Class: Truck)</i>	70
7.2	<i>CIFAR-10 classification confusion matrices (Part 1 of 2).</i>	71
7.2	<i>CIFAR-10 classification confusion matrices comparing the Baseline and optimized DARTS + Pruned architectures across all three tested topological families.</i>	72
7.3	Raspberry Pi terminal output showing real-time benchmarking progress	73
7.4	Comprehensive comparison of MobileNetV2-based models.	74
7.5	Radar chart illustrating normalized mobilenetv3 model resource.	74
7.6	Performance comparison of MobileNetV3-based architectures.	74
7.7	RPerformance comparison of MobileNetV2-based architectures.	75

LIST OF ABBREVIATIONS

API	Application Programming Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DARTS	Differentiable Architecture Search
DNN	Deep Neural Network
FLOPs	Floating Point Operations
FP32	32-bit Floating Point Precision
GPU	Graphics Processing Unit
INT8	8-bit Integer Precision
IoT	Internet of Things
MAC	Multiply-Accumulate Operation
NAS	Neural Architecture Search
ONNX	Open Neural Network Exchange

SYMBOLS AND NOTATIONS

α	Learning rate
$\hat{y}$	Predicted output
θ	Model parameters
b	Bias term of the neural network
L	Loss function
w	Weight parameters of the neural network
x	Input data
y	Ground truth label

CHAPTER 1

INTRODUCTION

What makes this situation even more depressing is when you train a deep learning model using the Raspberry Pi for the very first time. The machine-learning model that was trained successfully on a T4 GPU now takes up to 4ms on the Pi to give results, which was previously done within just 0.17ms, and all because the models were not designed to be executed on such hardware.

This project handles the exact problem. Edge AI the practice of performing inference on locally and on low power hardware, is very real and becoming more important. Autonomous devices, smart sensors, and Internet-of-Things devices need models that can make decisions very quickly using little power, without connecting to the internet. The hardware for these devices is extremely capable, but cannot control: low RAM, no dedicated GPU, and produced heat. Standard models like EfficientNet and MobileNet have already replaced some of the more heavyweight traditional models, but they are still general. When these are applied to an application on a target board, they often end up carrying computational overhead they do not need.

The approach we used was to start from scratch, but not to build from a human designed architecture, but rather a search based one. Differentiable Architecture Search (DARTS) treats the task of determining which operations to include in each layer of a network as a continuous optimization problem. Instead of relying on human insight to determine whether to use a 3x3 convolution or a depthwise separable convolution at a particular layer, the algorithm automatically learns the optimal configuration from data. Once an architecture is discovered, it is pruned using structural L1-norm pruning to remove non contributing channels, and then fine-tuned to maintain accuracy. We verified all of them on two platforms, Google Colab and Raspberry Pi 4. Comparison between MobileNetV2, MobileNetV3 and EfficientNet with our DARTS pipeline.

1.1 BACKGROUND

The history of deep learning was a trade off between accuracy and model size. First AlexNet, then VGG replaced it, then ResNet. With each models came more layers, parameters and compute, requiring hardware like multiGPU servers and memories. Even with such powerful hardware, any deployment smaller than that was unrealistic.

This resulted in a series of very light architectures. MobileNetV2 combined inverted residuals and linear bottlenecks. This brought a significant decrease in parameters while maintaining model accuracy. MobileNetV3 took it further by introducing neural architecture search to automate fine tuning, using hard swish activation and squeeze and excitation blocks. Efficient architectures was presented with EfficientNet which compounds network width, depth and resolution to achieve comparable accuracy over a range of model sizes.

All these architectures progressed in terms of speed and efficiency. However, these architectures are all designed to handle ImageNet, a database containing 1.2 million 224x224 pictures divided in 1000 classes. In this task, CIFAR-10, the database we work with, there are 60,000 32x32 pictures divided into 10 classes. The decision to build wide features, use a deep network and high-resolution stems makes no sense for a much simpler task like classification on CIFAR-10. A model built to overperform on a hard task will always be too much for a smaller task.

Neural Architecture Search (NAS) is a paradigm designed to automate network construction. Early methods based on reinforcement learning could take thousands of hours on several GPUs. The development of DARTS brought an end-to-end, differentiable solution that allows to perform architecture search in a matter of GPU-hours, by relaxing discrete search space into a weighted combination that can be optimized using gradient-descent. This way, architecture search is accessible without large machines clusters.

Model pruning also has a long history. Unstructured pruning consisted in removing single weights to obtain a sparse matrix that was often difficult to speed up in practice

without specific libraries. The structured pruning completely removes whole channels or filters from the model, yielding smaller, denser, but faster models executable on common hardware. In this project we applied the structured L1-norm pruning, which ranks the channels according to the size of the weights.

1.2 MOTIVATION

The primary motivation behind this project is fairly selfish: almost every paper that produces an efficient model lists the number of FLOPs and parameters. Few actually reports inference time on the type of hardware for which their model will be run. It's true that a model that's small on paper may run slow on a Raspberry Pi because of memory access patterns, cache effects or poor quantization compatibility. We wanted to investigate what really happens when these models hit the silicon, not just what the theoretical results predict.

The secondary motivation for this work is that the literature on efficient models is almost always split: neural architecture search and model pruning are almost always dealt with separately, and in isolation. Typically NAS papers produce good architectures which are then reported, pruning papers take some architecture (or hand designed architectures) and remove computation and parameters, and few tools can treat the entire search-prune workflow as one unified process. We theorized that an architecture searched and then subsequently pruned could obtain greater efficiency than either technique alone, and our results demonstrate this to be the case.

Lastly, there's the resources argument. Running inference on a Pi is cheap, causes little heat generation and requires no network connection, but only if it can run in a practical timeframe. An architecture that takes 10ms per image is not feasible for an application needing real-time vision, an architecture that takes under 1ms is. We investigate the range between these numbers.

1.3 SCOPE OF THE PROJECT

This project spans the entire workflow from model design to physical deployment. The specific problem we focus on is image classification on the CIFAR-10 dataset. It is small enough to be easily traceable. But enough to expose the inefficiency in general purpose models.

We implemented DARTS to search for an optimal cell structure, trained three different models (MobileNetV2, MobileNetV3 and EfficientNet) as baselines. Then pruned both DARTS discovered and baseline architectures with structured L1-norm pruning, followed by finetuning for accuracy recovery. We also tried INT8 quantization as a post-pruning compression technique for deployment testing.

Hardware-wise, we search and trained models on Google Colab and deployed on a Raspberry Pi 4. These two platforms represent two importantly different settings: one focusing on speed of training, one being closer to a real edge deployment scenario.

We evaluate the models using four main criteria: Top-1 accuracy, number of parameters, MACs and inference latency . Throughput and speedup ratio are then calculated as secondary metrics. We did not perform training on ImageNet, or in multi-task settings, and we did not explore on device training. This project is focused on the optimization of the inference step.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

The core problem that we address is whether a model searched for its architecture and then pruned can outperform a manually designed network on edge device.

To address this question we have developed a 'search-then prune' pipeline. The DARTS searches through combinations of operations within a specified cell structure. Then searches for the arrangement that performs best given the constraints imposed by the task of classification on CIFAR-10 and computational limits. This architecture is then subject to structured pruning. Then it is fine tuned to bring the accuracy back up. The final architecture is then exported and compared against pruned and unpruned MobileNetV2, MobileNetV3 and EfficientNet models.

The overall aim of the project was to highlight whether using search plus structured compression can result in architectures and performance profiles that manually designed models cannot achieve particularly when targeting a fixed hardware platform.

A secondary aim of this project was to examine where quantisation helps, and where it does not. By quantising to INT8, it became evident through the results on the Raspberry Pi that calculations involving FLOPs would never uncover. That quantization actually added latency rather than reduced it, potentially due to the cost of the quantization routines on a CPU lacking specialist INT8 acceleration. These kinds of results are only ever possible through actual deployment on the target hardware.

2.1 LITERATURE REVIEW

The initial CNN-based classifiers, AlexNet, VGG and ResNet proved that deep networks could perform feature learning much more effectively than manually engineered pipelines. However, these networks were very large, computationally expensive and assumed access to a GPU. The community thus generated a line of light weight architectures, built with the constraints of edge devices in mind. The inverted residual block in MobileNetV2 applies lightweight depthwise convolution in an expanded space and projects back down. It dramatically reduces parameter and FLOP counts over stan-

standard convolution. Neural architecture search was used in MobileNetV3 in conjunction with hard-swish activation and efficient Squeeze and Excite blocks to fine-tune network configuration and bring down parameters further. EfficientNet focused on compound scaling, balancing depth, width and resolution simultaneously instead of independently, making edge deployments more feasible. The above networks were however designed and tested on ImageNet.

Certain design choices are inefficient in the lower resolution / lower number of classes environment of CIFAR-10. Neural Architecture Search has attempted to remove human design decisions. The initial RL based NAS work by Zoph and Le showed machines designing better than humans, however, it was not computationally feasible. DARTS (Liu et al. 2019) has relaxed discrete architecture choices to a continuous space that can be optimised with gradient descent. On the pruning side, Han et al. Have showed that most weights can be removed with insignificant degradation in accuracy. However, the matrices generated by unstructure pruning cannot easily be exploited efficiently by existing hardware. Structured pruning removes whole filters or channels, resulting in dense architectures and enabling efficient execution on CPUs without specialized runtime. The NAS + pruning pipeline has yet to be explored in full on.

Table 2.1: Literature Review of DARTS-based Neural Architecture Search and Optimization Methods

Author	Year	Title	Methodology	Key Findings	Limitations
Liu et al. (2019)	2019	DARTS: Differentiable Architecture Search	Relaxes the discrete search space to a continuous one, enabling gradient-based joint optimisation of architecture and weights via bi-level optimisation.	Reduces search cost to ~ 4 GPU-days on CIFAR-10; achieves 2.76% test error, competitive with RL- and evolution-based NAS.	Prone to performance collapse due to skip-connection dominance; bi-level optimisation can be unstable and memory-intensive.

Author	Year	Title	Methodology	Key Findings	Limitations
Zoph & Le Zoph and Le (2017)	2017	Neural Architecture Search with RL	Uses a recurrent controller trained with REINFORCE to generate and evaluate neural network architectures on a validation set.	Discovers novel architectures that outperform hand-designed models on CIFAR-10 and Penn Tree-bank language modelling.	Requires 800 GPUs over 28 days; prohibitively expensive for most research groups; limited transferability.
Pham et al. Pham et al. (2018)	2018	ENAS: Efficient NAS via Parameter Sharing	Shares weights across all child models within a supernet-work, replacing independent training with a single shared optimisation.	Reduces search to ~ 0.45 GPU-days; achieves 2.89% error on CIFAR-10 with dramatically lower cost than standard NAS.	Weight sharing introduces coupling bias; architectures found may not generalise as well as those trained independently.
Chen et al. Chen et al. (2019)	2019	P-DARTS: Progressive DARTS	Progressively increases search network depth across stages while pruning the operation space to reduce computational overhead.	Achieves 2.50% error on CIFAR-10 in 0.3 GPU-days; stronger generalisation to ImageNet than standard DARTS.	Progressive pruning risks eliminating beneficial operations early; depth increase raises GPU memory requirements per stage.

Author	Year	Title	Methodology	Key Findings	Limitations
Xu et al. Xu et al. (2020)	2020	PC-DARTS: Partial Channel Connections	Samples a subset of channels per edge during search to reduce memory and enable direct architecture search on ImageNet.	Finds competitive architectures directly on ImageNet in ~ 3.8 GPU-hours; 24.2% top-1 error without proxy datasets.	Channel sampling introduces randomness that may hurt search stability; sensitive to sampling ratio hyperparameter.
Xie et al. Xie et al. (2019)	2019	SNAS: Stochastic NAS	Uses Gumbel-Softmax to sample discrete architecture decisions in a differentiable manner, avoiding the bi-level optimisation of DARTS.	Single-level optimisation is more principled than DARTS; achieves 2.85% error on CIFAR-10 with lower search time.	Gumbel temperature annealing requires careful scheduling; harder to reproduce stable results across different random seeds.
Cai et al. Cai et al. (2019)	2019	ProxylessNAS: Direct NAS on Target Hardware	Binarises architecture parameters to train only two active paths at a time, enabling memory-efficient search directly on target datasets and hardware.	Achieves hardware-aware search on ImageNet and mobile platforms; outperforms MobileNetV2 in latency-accuracy trade-offs.	Binarisation introduces gradient estimation error; result quality depends strongly on the choice of hardware latency lookup table.

Author	Year	Title	Methodology	Key Findings	Limitations
Chu et al. Chu et al. (2020)	2020	Fair DARTS: Eliminating Unfair Advantages	Replaces softmax with independent sigmoid outputs per operation, removing the exclusive competition that favours skip connections.	Eliminates skip-connection dominance; achieves 2.54% error on CIFAR-10 with more balanced and interpretable operation selection.	Sigmoid relaxation weakens inter-operation competition; may lead to redundant operations being selected together on some edges.
Zela et al. Zela et al. (2020)	2020	Understanding and Robustifying DARTS	Identifies a correlation between dominant Hessian eigenvalue of validation loss and generalisation failure; proposes regularisation to constrain Hessian norm during search.	Diagnoses root cause of DARTS collapse; regularised variants achieve stable search across multiple challenging search spaces.	Hessian computation adds overhead; proposed regularisation methods vary in effectiveness across different search spaces.

Author	Year	Title	Methodology	Key Findings	Limitations
Liang et al. Liang et al. (2019)	2019	DARTS+: Improved DARTS with Early Stopping	Introduces an early-stopping criterion triggered when skip connections begin to dominate, preventing optimisation collapse.	Achieves 2.32% on CIFAR-10 and 23.7% top-1 on ImageNet; simple and effective over vanilla DARTS without architectural changes.	Early stopping threshold requires manual tuning; does not address the underlying instability of bi-level optimisation in DARTS.
Chu et al. Chu et al. (2021)	2021	DARTS ⁻ : Robustly Stopping Out of Collapse	Avoids performance collapse by using an auxiliary skip-connection bypass and an operation-level auxiliary loss during search.	Consistently robust across multiple search spaces; achieves 2.59% on CIFAR-10 without any early stopping or Hessian regularisation.	Auxiliary connections add parameters to the super-net during search; slight increase in search memory compared to vanilla DARTS.

Author	Year	Title	Methodology	Key Findings	Limitations
Ye et al. Ye et al. (2022)	2022	β -DARTS: Beta-Decay Regularisation	Applies Beta-Decay regularisation to constrain the magnitude and variance of architecture parameters, ensuring fairer operation competition throughout the search process.	Stabilises search across diverse search spaces and datasets; outperforms prior regularisation-based DARTS variants consistently.	Introduces additional hyperparameter β ; effectiveness of decay schedule is sensitive to the search space configuration.
Wang et al. Wang et al. (2021)	2021	Rethinking Architecture Selection in DARTS	Challenges the reliability of architecture parameters as selection indicators; proposes perturbation-based architecture selection as a more robust alternative.	Demonstrates that argmax on architecture weights is unreliable; perturbation selection yields more accurate and stable architectures.	Perturbation-based selection increases post-search computational cost; not easily integrated into fully differentiable pipelines.

Author	Year	Title	Methodology	Key Findings	Limitations
Real et al. Real et al. (2019)	2019	Regularised Evolution for Architecture Search	Uses evolutionary search with tournament selection and weight inheritance, applying aging regularisation to remove the oldest models from the population.	Discovers AmoebaNet achieving 3.34% on CIFAR-10 and 83.9% top-5 on ImageNet; outperforms earlier evolutionary and RL methods.	Requires 3,150 GPU-days despite efficiency improvements; evolution-based search is still significantly more expensive than DARTS.
Liu et al. Liu et al. (2018)	2018	PNAS: Progressive Neural Architecture Search	Uses a sequential model-based optimisation strategy guided by a surrogate predictor to evaluate architectures in order of predicted performance.	Reduces the number of model evaluations by $5\times$ versus NAS-RL while achieving comparable accuracy on CIFAR-10 and ImageNet.	Surrogate predictor quality is critical; progressive search is sequential and cannot be easily parallelised across GPU clusters.
Dong & Yang Dong and Yang (2020)	2020	NAS-Bench-201: Extended NAS Benchmark	Provides a cell-based benchmark with a fixed search space of 15,625 architectures evaluated across CIFAR-10, CIFAR-100 and ImageNet-16.	Enables fair comparison of NAS algorithms across three datasets; reveals that many methods fail to outperform random search reliably.	Fixed cell topology limits evaluation of non-cell-based search spaces; benchmark may not reflect real-world deployment constraints.

Author	Year	Title	Methodology	Key Findings	Limitations
Chen & Hsieh Chen and Hsieh (2020)	2020	Stabilising DARTS via Perturbation Regularisation	Adds perturbation-based regularisation (SDARTS) during the architecture parameter update step to smooth the loss landscape.	Significantly improves search stability; achieves 2.61% on CIFAR-10 with consistent results across runs and search spaces.	Perturbation increases per-step computation; regularisation strength requires careful tuning for different tasks and datasets.
Guo et al. Guo et al. (2020)	2020	Single Path One-Shot NAS	Decouples architecture search from weight training by uniformly training a single-path supernet, then applying evolutionary search to find optimal sub-architectures.	Achieves 25.3% top-1 error on ImageNet in a two-stage pipeline; avoids weight-coupling issues inherent in DARTS-style approaches.	Uniform path sampling may under-represent rare but high-quality architectures; evolutionary search phase adds separate cost.

Author	Year	Title	Methodology	Key Findings	Limitations
Tan & Le Tan and Le (2019)	2019	EfficientNet: Model Scaling for CNNs	Uses neural architecture search to find a baseline architecture, then scales depth, width and resolution jointly using a compound coefficient derived from a constrained optimisation problem.	EfficientNet achieves 84.3% top-1 on ImageNet with $8.4\times$ fewer parameters and $6.1\times$ fewer FLOPs than ResNet-152.	Compound scaling assumes fixed ratio between depth, width and resolution; may not hold for non-standard tasks or architectures.
Sandler et al. Sandler et al. (2018)	2018	MobileNetV2: Inverted Residuals & Linear Bottlenecks	Introduces inverted residual blocks with linear bottlenecks and depthwise separable convolutions, with NAS principles informing the block design for mobile efficiency.	Achieves strong ImageNet accuracy with $\sim 3.4\text{M}$ parameters; used as backbone and baseline in many subsequent NAS methods.	Linear bottleneck assumption limits non-linearity; gains over MobileNetV1 require careful learning rate and regularisation tuning.

Author	Year	Title	Methodology	Key Findings	Limitations
Howard et al. Howard et al. (2019)	2019	MobileNetV3: Searching for MobileNets	Combines hardware-aware NAS with the NetAdapt algorithm and manual refinements (h-swish activation, SE modules) to tune architectures for mobile CPU latency.	MobileNetV3-Large achieves 75.2% top-1 on ImageNet at 51M multiply-adds; 20% faster than MobileNetV2 at similar accuracy.	NAS component requires significant hardware profiling infrastructure; h-swish activation may not be optimal for all deployment targets.

2.2 RESEARCH GAP

Major research gaps that motivated this work:

1. Integration between NAS and Pruning: NAS) and pruning are usually two separate areas of research. While the DARTS-based works concentrate on finding high accuracy architectures, no further analysis regarding compression or execution on edge devices is made. At the same time, pruning algorithms work only on predefined architecture without trying to find the best possible architecture via NAS methods. Evaluation of performance of architectures found by NAS combined with structured pruning approach and validation on edge hardware has not been explored well enough.

2. Disconnect between proxy metrics and real performance: Standard proxy metrics for performance comparison such as number of FLOPs and number of parameters are often a very bad estimator of actual latency of models on edge devices. Experimental results show that on Raspberry Pi INT8 quantized model performs worse than its FP32 variant. It happens due to lack of hardware support of INT8 operations in CPUs such as ARM Cortex-A72 in this case quantization costs more than it saves from performance perspective. Such an issue cannot be detected if the assessment is purely theory based.

3. Little Analysis Done on Edge Platforms: A major part of evaluation for edge AI

involves use of highly efficient platforms that have GPU-based acceleration capabilities. On the contrary, there are other devices like the Raspberry Pi which can be used as an example in terms of practical deployment. Such systems lack accelerator support, memory constraints, and work with CPUs only.

2.3 OBJECTIVES

The primary objectives of this project are as follows:

1. Design and develop a unified pipeline which integrates DARTS Neural Architecture Search (NAS) and structured pruning in one end-to-end procedure.
2. Develop and test the performance of baseline models. This includes MobileNetV2, MobileNetV3, and EfficientNet to perform image classification on CIFAR-10.
3. Deploy all versions of the models on Raspberry Pi to measure latency on the edge devices instead of theoretical proxy measurements such as the number of operations performed by the model.
4. To analyze and visualize the trade-off between model accuracy and inference latency, providing practical insights for deploying computer vision models on CPU-only edge devices.
5. To investigate the impact of post-training quantization (INT8) on CPU-based inference, identifying scenarios where it improves performance and cases where it introduces computational overhead.

2.4 PROBLEM STATEMENT

The first statement is simple. Models that perform efficiently during training do not always perform efficiently on edge devices. Attempting to reduce this gap by employing an efficient, general-purpose model but hoping that it works efficiently on the hardware results in inefficiencies. The MobileNetV2 baseline requires 4.37ms per image on the Raspberry Pi; our DARTS+Pruning model requires 0.78ms, for a 5.57x speedup while achieving within 2% it wasn't built for this particular problem/hardware, but rather a general problem and a general hardware platform that was then applied here.

2.5 PROJECT PLAN

The project was divided into four phases:

Phase 1: Environment Setup and Literature Review We set up the environment for the search and training in Google Colab and for the deployment on Raspberry Pi 4. One main constraint was that model format (ONNX) exporting from Colab should work properly on the Pi without dependency issues. We have surveyed relevant papers on DARTS, structured pruning and edge deployment to determine hyperparameter values before running any experiments.

Phase 2: Architecture Search and Baseline Training -

We have searched the cell architecture by running DARTS on CIFAR-10 and trained baseline models MobileNetV2, MobileNetV3 and EfficientNet using the same training recipe in Colab. DARTS searched a network much smaller than all three baseline networks in parameters (0.02M vs MobileNetV2 with 2.24M) but higher in accuracy (82.59% vs 70.94%).

Phase 3: Pruning and Fine-Tuning -

We have performed structured L1-norm pruning on all models, each by removing the bottom proportion of the channel with the smallest L1-norm in each conv layer. Then, fine-tuned models by training for a number of steps or epochs using lower learning rates. Besides pruning, we also applied INT8 post-training quantization as an additional step and evaluated both FP32 and INT8 models on Pi.

Phase 4: Deployment and Evaluation

All trained and pruned models have been uploaded to Raspberry Pi and evaluated for inference latency by taking the average value of several times, considering that latency measurement may vary a lot. All results for accuracy, latency, throughput, speedup and model size have been collected and analyzed for comparison.

#	Phase	Key Activities	Outcomes
1	Environment Setup & Literature Review	Configured Colab and Raspberry Pi 4; reviewed DARTS and pruning literature.	Validated ONNX pipeline; hyperparameters selected.
2	Dataset Preparation	Preprocessed CIFAR-10 with augmentation (crop, flip, normalization); verified data loaders.	Reproducible pipeline ready for all models.
3	Baseline Model Training	Trained MobileNetV2, MobileNetV3, and EfficientNet under identical settings.	Baseline accuracy and parameter counts recorded.
4	Neural Architecture Search (DARTS)	Ran DARTS on CIFAR-10; selected best cell; retrained full network from scratch.	0.02M parameter model at 82.59% accuracy; outperforms all baselines.
5	Structured Pruning & Fine-Tuning	Applied L1-norm channel pruning to all models; fine-tuned at reduced learning rate.	Smaller models with accuracy largely recovered post pruning.
6	Post-Training Quantization	Converted all pruned FP32 models to INT8 via ONNX PTQ; validated on test set.	INT8 models ready; accuracy drop vs. FP32 documented.
7	Edge Deployment	Deployed FP32 and INT8 ONNX models on Raspberry Pi 4; measured latency and throughput.	Latency, throughput, speedup, and model size benchmarked per model.
8	Analysis & Reporting	Compared all models on accuracy, latency, size, and efficiency trade-offs.	DARTS + INT8 identified as optimal edge configuration.

CHAPTER 3

TECHNICAL SPECIFICATION

Technical specifications of the system will be examined in this chapter. It will include architecture specification, constraints, and requirements. NAS and structured pruning will be elaborated on in this chapter to understand what technical specifications are needed for deployment. Technical specifications will be included in this chapter that ensure the scalability and efficiency of the system that can be deployed to perform inference at the edges.

3.1 REQUIREMENTS

In order to realize the objective that the entire proposed process of Neural Architecture Search(NAS) and structured pruning aims to achieve, it is necessary to have a very clearly defined set of hardware, software, and other requirements. For the process of NAS itself, it should automatically be able to detect the neural architectures and redundant channels that it will need.

3.1.1 Functional and Non-Functional Requirements

Functional Requirements

- **Dataset Pipeline:** Download the CIFAR-10 dataset. Apply data augmentation using RandomCrop and RandomHorizontalFlip. Normalize the dataset using channel-wise means (0.4914, 0.4822, 0.4465) and standard deviations (0.2023, 0.1994, 0.2010)
- **Differentiable Architecture Search (DARTS):** A NAS controller assesses a pre-defined set of candidate operations : Separable Convolutions, Dilated Convolutions, Identity, and Max Pooling via gradient descent-based evaluation.
- **Structured Pruning Engine:** Prune filters based on their L1-norm significance value and physically remove structurally redundant elements, pruning up to 50% of the expansion channels, strictly without modifying depthwise convolutions to avoid spatial dimension and group convolution inconsistencies.

- **Edge Translation:** Convert from a PyTorch dynamic graph (.pth) to an Open Neural Network Exchange static graph (.onnx) in both FP32 and INT8 (quantized) forms for deployment to edge devices.

Non-Functional Requirements

- **Edge Constraint Tolerance:** This is about the ability of the network that has been trimmed to be executable by edge devices (Raspberry PI) within the constraints of its hardware resources like memory and computing capability without going beyond the heat limit of the device.
- **Reproducibility:** Replicability: Random weight initialization, data splitting, algorithm parameters are all set to be based on one fixed seed value (e.g., seed = 42).
- **Latency Accuracy Trade-off:** Reductions in parameter count must yield measurable reductions in inference latency (in milliseconds) without catastrophic degradation of classification accuracy.

3.2 FEASIBILITY STUDY

This design is quite practical from a technical perspective. Modern deep learning libraries and available edge computing runtimes are used.

Technical Feasibility: PyTorch supports auto-differentiation that helps optimize network weights as well as alphas together. ONNX Runtime acts as an intermediary here. It allows PyTorch trained models, which cannot be easily executed natively, to be executed at the edge using the CPU hardware.

Feasibility of Computation: Conventional DARTS is highly resource-intensive. Constraining the initial search space to 6 layers and 16 channels and employing batch size limitations during search allows for the completion of training in a reasonable amount of time even when using affordable GPUs such as those provided by Google Colab.

Operational Feasibility: The automated and precise hardware benchmarking script provides validation of theoretical metrics, such as MACs derived from onnx-tool. It

evaluates system performance in terms of execution speed, thereby validating latency and frames per second.

3.3 SYSTEM SPECIFICATION

The pipeline architecture follows the sequential order having 5 stages to find, compress, and evaluate CNNs.

Baseline module: Training of a regular MobileNetV2/MobileNetV3-Small/ Efficient-Net model and registering benchmarks with respect to speed performance. This model will act as a base to measure the speed up.

NAS Controller module: Training of the DARTS-based search network with MixedOp layers. It uses two optimizers for training weights (SGD) and architectural parameters (Adam). In this way, it learns a discrete architecture which is optimized for the CIFAR-10 dataset.

Architecture Rebuilding & Pruning module: It first extracts the discrete architecture learned by the NAS Controller and rebuilds the architecture using the physical method. Then it uses L1-norm criteria for channel pruning to get pruning masks that lead to the compression of the architecture. Finally, Batch Normalization and Linear layers are tuned to be compatible with the width of the pruned convolutions.

Fine-Tuning module: Fine-tuning of the pruned architecture using Adam optimizer with CosineAnnealingLR scheduler. It applies label smoothing regularization to avoid overconfidence due to pruning, hence generalizing the architecture.

Deployment & Profiling module: Exporting the trained model in ONNX format and setting up an inference session. The final evaluation includes Parameters (Params), Multiply-Accumulate Operations (MACs), Latency (ms), and Frames Per Second (FPS).

Table 3.1: Hardware and Software Environment Specifications

Component / Phase	Specification / Detail
Cloud Training Environment	Google Colab
Training Hardware	NVIDIA GPU (CUDA Enabled)
Edge Hardware (Target)	Raspberry Pi / Jetson Nano
Operating System	Linux (Ubuntu)
Programming Language	Python 3
Deep Learning Framework	PyTorch, Torchvision
Edge Deployment Runtime	ONNX Runtime (CPUExecutionProvider)
Profiling & Metrics Tools	thop (FLOPs), onnx-tool (MACs)

Table 3.2: Comprehensive Hyperparameter Configurations

Pipeline Phase	Parameter	Value / Configuration
Global Settings	Dataset	CIFAR-10
	Batch Size	128
	Random Seed	42
1. Baseline Training	Epochs	50
	Optimizer	SGD (LR: 0.1, Momentum: 0.9)
	Weight Decay	5×10^{-4}
	LR Scheduler	MultiStepLR (Milestones: 25, 40)
2. DARTS Search	Search Space	6 Layers, 16 Initial Channels
	Search Epochs	50 (Max 50 batches/epoch)
	Weight Optimizer	SGD (LR: 0.025, WD: 3×10^{-4})
	Architecture Optimizer	Adam (LR: 3×10^{-4} , WD: 1×10^{-3})
3. Structured Pruning	Pruning Criterion	L1-Norm Ranking
	Prune Amount	50% (0.5) of Expansion Channels
	Network Constraints	Skip Depthwise, Skip First/Last Layers
4. Fine-Tuning	Epochs	50
	Optimizer	Adam (LR: 1×10^{-3})
	LR Scheduler	CosineAnnealingLR
	Regularization	Label Smoothing ($\epsilon = 0.1$)

CHAPTER 4

SYSTEM DESIGN

In this section, we will detail the design of the system architecture. The correlation between NAS and structured pruning is discussed in this chapter as well. The relationship among each of the modules while running the system and its significance are discussed in this section.

4.1 SYSTEM ARCHITECTURE

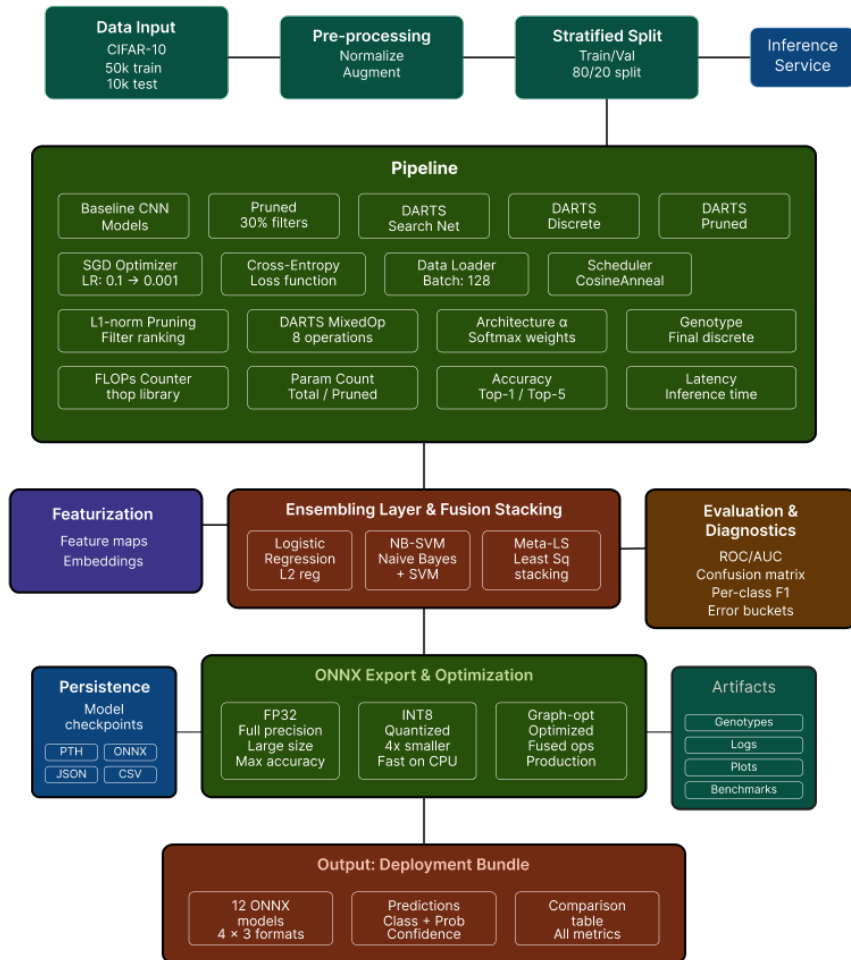


Figure 4.1: System Architecture

As illustrated in Figure 4.1, the pipeline begins with data ingestion and preprocessing, this system is designed to be a multistep pipeline for performing deep learning. In this case, the problem at hand is image classification by the CIFAR-10 dataset. The system workflow includes several stages in order to conduct image classification,

Data Input and Preprocessing: The 50,000 images in CIFAR-10 constitute the training images and the 10,000 images in CIFAR-10 are the testing images; CIFAR-10 has 10 image categories. Training images for the CNN models will undergo data augmentation and normalization procedures. Split is into 80% training set and 20% validation set.

Core Training Block: A block that trains the 5 models concurrently: baseline CNN, pruned CNN (30 filters removed), DARTS search network (with continuous relaxation), optimal model found by DARTS (discrete DARTS model), and pruned DARTS. These models use a similar training algorithm that comprises of: Stochastic gradient descent (SGD) with learning rate decayed from 0.1 to 0.001, cross-entropy loss function, 128 batch size, and cosine annealing schedule.

Neural Architecture Search via DARTS: This involves converting discrete search into a continuous optimization task using the DARTS methodology. During the search phase, each edge within the network is investigated for its performance using eight operators; these could be convolutional, skip, or pooling operations. By the end of the search phase, the optimal operation will have the highest weight within each of its edges, revealing the optimal genotype. Next, the DARTS generated architecture is optimized using structured pruning; specifically using L1 norm for filter ranking.

Featurization and Ensemble Learning: After training, intermediate features/embeddings are generated from each model variant and are fed into an ensemble layer. An ensemble strategy based on stacking is applied where results from the 5 models are stacked using three different meta-classifiers: Logistic Regression classifier (with L2 regularization), NB-SVM classifier, and Least squares stacker. The use of ensembling strengths each model to optimize the performance of the overall system.

Evaluation and Diagnostics: A wide range of measures were used to evaluate each model variation, these measures include ROC/AUC, confusion matrix, F1-score per class, error bucketing, etc., to gain a global view and a detailed insight into each model’s performance.

Persistence and ONNX Export: All trained models are stored using PyTorch checkpoints (.pth), ONNX models, and structured CSV/JSON logs showing genotype, and all measurements and parameters. They are stored in three different formats: FP32 (full precision server inference), INT8 (for low-resource environments where the model is significantly smaller and optimized), and graph-optimization optimized ONNX.

Deployment Bundle: All prediction models are bundled with prediction results based on the probability score for each class, a table summarizing F1 scores, float-point operations (FLOPs), parameter count, inference time, etc. For each model and ONNX version, and 12 individual ONNX files for each of the four configurations (3 ONNX versions for 4 different architectures).

4.2 DATA FLOW DIAGRAM

Figure 4.2 All the operating procedures for the initial three simplified models is captured by the single data flow diagram of the whole operating system, shown in Figure 4.2. The pipeline starts at a single node as all three models use the ImageNet pretrained weight as a base to train the CIFAR-10 model after the classifier head was substituted with the new structure to produce 10 class output. The base training uses the same optimizer setting; SGD with momentum 0.9 and weight decay 5×10^{-4} , the optimizer is adjusted with MultiStepLR which decreases at epoch 25 and 40 for total of 50 epochs. Then the pipeline bifurcates into three branches based on the different pruning process for each model. L1-norm filter pruning is applied on the Conv2d layer which meets the eligible condition at ≥ 0.3 for MobileNetV2, and a new compressed model was generated through weight slicing via $W=W[\text{Iout}, \text{Iin}, :, :]$ way. MobileNetV3-Small function will only applied on the expansion 1 1 convolution in InvertedResidual blocks and operate at the maximal ratio of ≤ 0.5 while the block-level model reconstruction requires rescale the SE module. As for EfficientNet, its blocks are found using auto-detection

before pruning at $\alpha=0.3$ since all components in MBConv block need to be rebuilt such as expansion convolutions, depthwise convolutions, two Batchnorms, SE block and projection convolutions. Both Fine-tuning approaches on MobileNetV2 and EfficientNet share the same training procedure by using SGD and StepLR ($\gamma=0.5$ after 15 epoch) in 30 epochs whereas MobileNetV3-Small is tuned using Adam and CosineAnnealingLR with label smoothing $= 0.1$ in 50 epochs in terms of different weight distribution at high pruning rate.

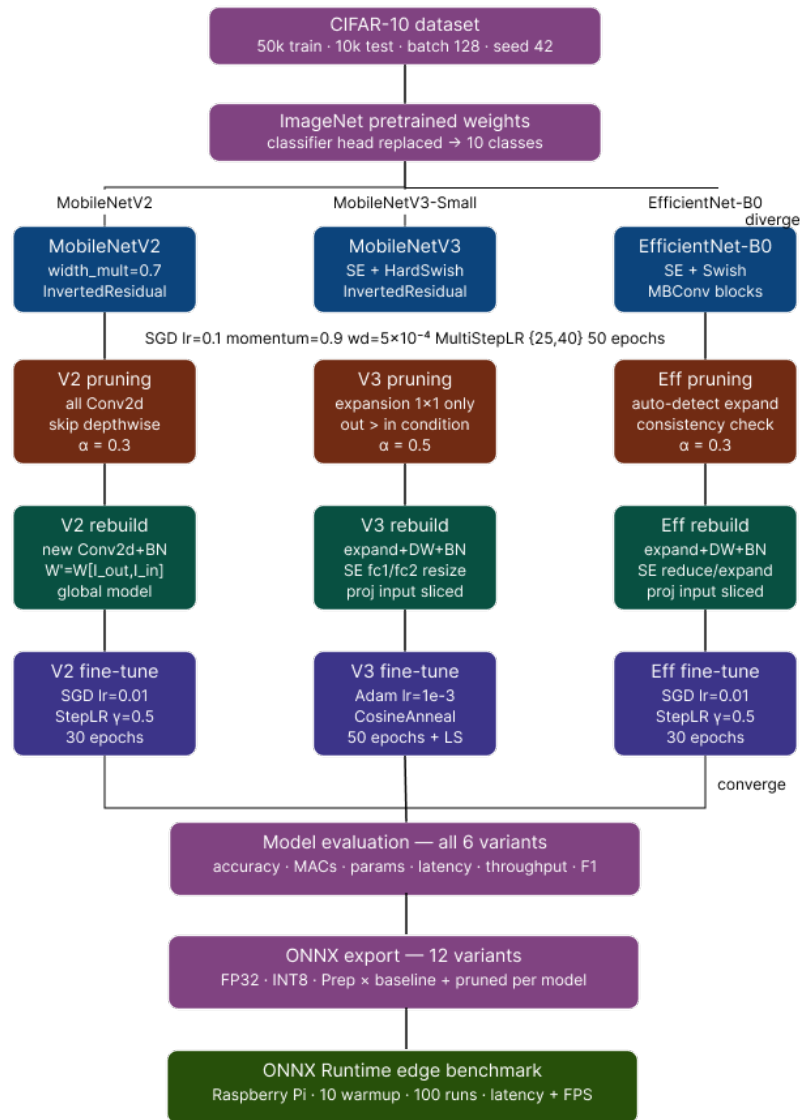


Figure 4.2: Consolidated data flow for all three baseline and pruned model pipelines

CHAPTER 5

METHODOLOGY AND TESTING

In this chapter, an explanation is given regarding the methodological approach used for designing, implementing, and evaluating the Neural Architecture Search and Structured Pruning techniques proposed. It also explains the methodology used to create and optimize neural networks through pruning techniques, while analyzing their performance.

5.1 METHODOLOGY

5.1.1 Overview

This project applies a structured pipeline that includes an existing model architecture MobileNetV2, MobileNetV3, EfficientNet, structured L1-norm pruning technique, differentiable neural architecture search (DARTS), and multi-format conversion to ONNX for edge devices in image classification tasks of CIFAR-10. Four models are trained in the pipeline and measured in terms of their accuracy, number of parameters, multiply-accumulate operations (MACs), inference time, and throughput. All experiments are done in a reproducible way using the same fixed random seed value of 42 throughout PyTorch, NumPy, and Python stdlib.

5.1.2 Dataset Preparation

The CIFAR-10 dataset is used as the benchmark. It consists of 60,000 colour images of size 32×32 pixels across ten object categories. 50,000 images form the training set and 10,000 form the test set. The training set is further divided into an 80% training subset and a 20% validation subset. This validation split is used exclusively during the DARTS architecture search phase for updating the architecture parameters; all other models are trained on the full 50,000-sample training set. Two separate transform pipelines are applied. During training, images are augmented using random cropping with a padding of 4 pixels and random horizontal flipping, followed by normalisation using the CIFAR-10 channel-wise mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2023, 0.1994, 0.2010). During evaluation and testing, only normalisation is applied with no augmentation. All data loaders use a batch size of 128 with two worker

processes and pinned memory for efficient GPU data transfer.

5.1.3 Baseline Models

The base models are initialized with weights pre-trained on ImageNet dataset. The last layer is omitted by a new `nn.Linear` layer which outputs 10 classes. The model uses a width multiplier value of 0.7, i.e., it reduces the number of channels per layer in order to decrease the number of parameters and computation complexity, while keeping the inverted residual blocks architecture of the original one. The optimizer selected for the model training process is SGD, with a momentum value of 0.9, weight decay of 5×10^{-4} , and an initial learning rate of 0.1. A MultiStepLR learning rate scheduler is used with a learning rate decay at epoch numbers 25 and 40 over a total period of 50 training epochs. The criterion used is the cross-entropy loss. The best performing model checkpoint is stored in file `baselinemodel.pth` and used as the base for all future comparisons.

5.1.4 Structured Pruning of Baseline Models

Structured filter pruning is applied to the trained baseline models to reduce their parameter count and computational cost. The pruning procedure uses L1-norm filter ranking: for each eligible convolutional layer, the L1-norm of each filter’s weight tensor is computed, and the lowest-ranked 30% of filters are identified for removal. Depthwise convolutional layers (those with `groups > 1`) are skipped to preserve the integrity of the inverted residual blocks, and the final convolutional layer is also excluded to avoid disrupting the classification head. The stem layer (`features.0.0`) is included in the pruning scope.

After obtaining masks for all the layers, a new compact model is created by resetting the channel dimension according to the remaining filters. The convolutional stem layer is then created based on the number of output channels that survived the process. Its parameters are simply copied from the original model. The other nonpruned layers are copied directly from the baseline model through deep copy. The compact model is first checked via a dummy forward pass to validate its architecture prior to fine-tuning for 30 epochs. The process employs SGD optimization algorithm with a learning rate of 0.01, momentum of 0.9, weight decay of 5×10^{-4} , and step-wise learning rate reduction

every 15 epochs.

5.1.5 Neural Architecture Search: DARTS

Search Space and Operations

The DARTS search space defines a six-layer network where each layer is represented as a *MixedOp*: a weighted combination of five candidate operations. These include:

- 3×3 Separable Convolution (sep_3x3)
- 5×5 Separable Convolution (sep_5x5)
- 3×3 Dilated Convolution with dilation rate of 2 (dil_3x3)
- Identity (with 1×1 projection in case of different channel sizes)
- 3×3 Max Pooling with 1×1 convolution (max_pool)

. The network begins with a stem convolution of 16 initial channels, and the channel count doubles at layers placed at one-third and two-thirds of the network depth, where the stride is set to 2 to spatially downsample the feature maps.

Bi-Level Optimisation

The DARTS search network jointly optimises two sets of parameters: the network weights w and the architecture parameters α :

$$\min_{\alpha} \mathcal{L}_{\text{val}}(w^*(\alpha), \alpha) \quad \text{s.t.} \quad w^*(\alpha) = \arg \min_w \mathcal{L}_{\text{train}}(w, \alpha) \quad (5.1)$$

In practice, at each search step the network weights are updated on a training batch using SGD with learning rate 0.025, momentum 0.9, and weight decay 3×10^{-4} ; the architecture parameters are then updated on a separate validation batch using Adam with learning rate 3×10^{-4} , $\beta = (0.5, 0.999)$, and weight decay 1×10^{-3} . The forward pass through each MixedOp computes a softmax-weighted sum of the outputs of all five operations, where the softmax is applied over the corresponding α vector for that layer. The search runs for 50 epochs, processing 50 mini-batches per epoch to limit search time.

Architecture Discretisation

Following the search process, the discrete genotype is obtained through choosing the operation which has the largest α in each of the six levels. The genotype is a list of operations (*e.g.*, [`'sep_3x3'`, `'max_pool'`, ...]). The genotype file is stored together with the checkpoint file of the search network.

Training the Final DARTS Network

A clean `DARTSFinalNetwork` is constructed from the discovered genotype. The network uses the same stem and layer structure as the search network but replaces each `MixedOp` with the single selected operation. It is trained from scratch for 50 epochs using SGD with an initial learning rate of 0.025, momentum 0.9, weight decay 3×10^{-4} , and a `MultiStepLR` scheduler decaying by 0.1 at epochs 25 and 40. The best checkpoint (by validation accuracy) is saved as `darts_final_network.pth` along with the genotype and accuracy.

5.1.6 DARTS Network Pruning

Structured pruning is applied to the trained DARTS Final Network following the same L1-norm filter ranking procedure used for the CNNs, with a pruning ratio of 30%. The pruning implementation traverses all `nn.Conv2d` layers in the DARTS network, computes per-filter L1-norms, and identifies filters to remove. A compact DARTS model is then rebuilt by replacing each pruned convolution with a new `nn.Conv2d` layer of reduced output channels, copying the surviving filter weights, and propagating the channel changes through the subsequent batch normalisation and convolutional layers. The pruned DARTS model is fine-tuned for 30 epochs using SGD with a learning rate of 0.01 and weight decay of 1×10^{-4} , and saved as `darts_pruned_finetuned.pth`.

The Differentiable Architecture Search pipeline is organised into five sequential phases as illustrated in Figure 5.1. In Phase 1, a `DARTSSearchNetwork` is constructed with a 3×3 stem convolution producing 16 channels, followed by six *MixedOp* layers each representing a softmax-weighted combination of five candidate operations — 3×3 and 5×5 separable convolutions, 3×3 dilated convolution with dilation rate 2, identity, and 3×3 max pooling followed by a 1×1 convolution. Each layer maintains a learn-

able architecture parameter vector $\alpha \in \mathbb{R}^5$, initialised as $10^{-3} \cdot \mathcal{N}(0, 1)$, and the channel count doubles at layers 2 and 4 where the stride is set to 2, progressing from 16 to 32 to 64 channels. In Phase 2, the network undergoes bi-level optimisation over 50 epochs of 50 mini-batches each. For all iterations, the neural network’s weights w will be optimized using SGD with a learning rate of 0.025, momentum of 0.9, and a weight decay factor of 3×10^{-4} for the training mini-batch, while optimizing the architecture parameters α with the Adam optimizer having a learning rate of 3×10^{-4} , $\beta = (0.5, 0.999)$, and weight decay 1×10^{-3} , minimizing $\min_{\alpha} \mathcal{L}_{\text{val}}(w^*(\alpha), \alpha)$. For Phase 3, the discrete genotype will be generated through

$$\text{genotype}[i] = \arg \max(\text{softmax}(\alpha_i))$$

on six layers to output a sequence of six operation types. In Phase 4, a clean DARTS Final Network is built from this genotype by replacing each *MixedOp* with its single selected operation, and trained from scratch for 50 epochs using SGD with MultiStepLR decaying by $\gamma = 0.1$ at epochs 25 and 40, saved as `darts_final_network.pth`. In Phase 5, the trained final network is subjected to structured L1-norm pruning at a ratio of $\alpha = 0.3$, with per-filter importance scores $s_i = \|\mathbf{W}_i\|_1$ computed for all eligible Conv2d layers, depthwise layers skipped, and surviving weights sliced as $\mathbf{W}' = \mathbf{W}[\mathcal{I}_{\text{out}}, \mathcal{I}_{\text{in}}, :, :]$ into a compact model validated by a dummy forward pass. The pruned model is fine-tuned for 30 epochs using SGD with a learning rate of 0.01 and weight decay 1×10^{-4} , saved as `darts_pruned_finetuned.pth`.

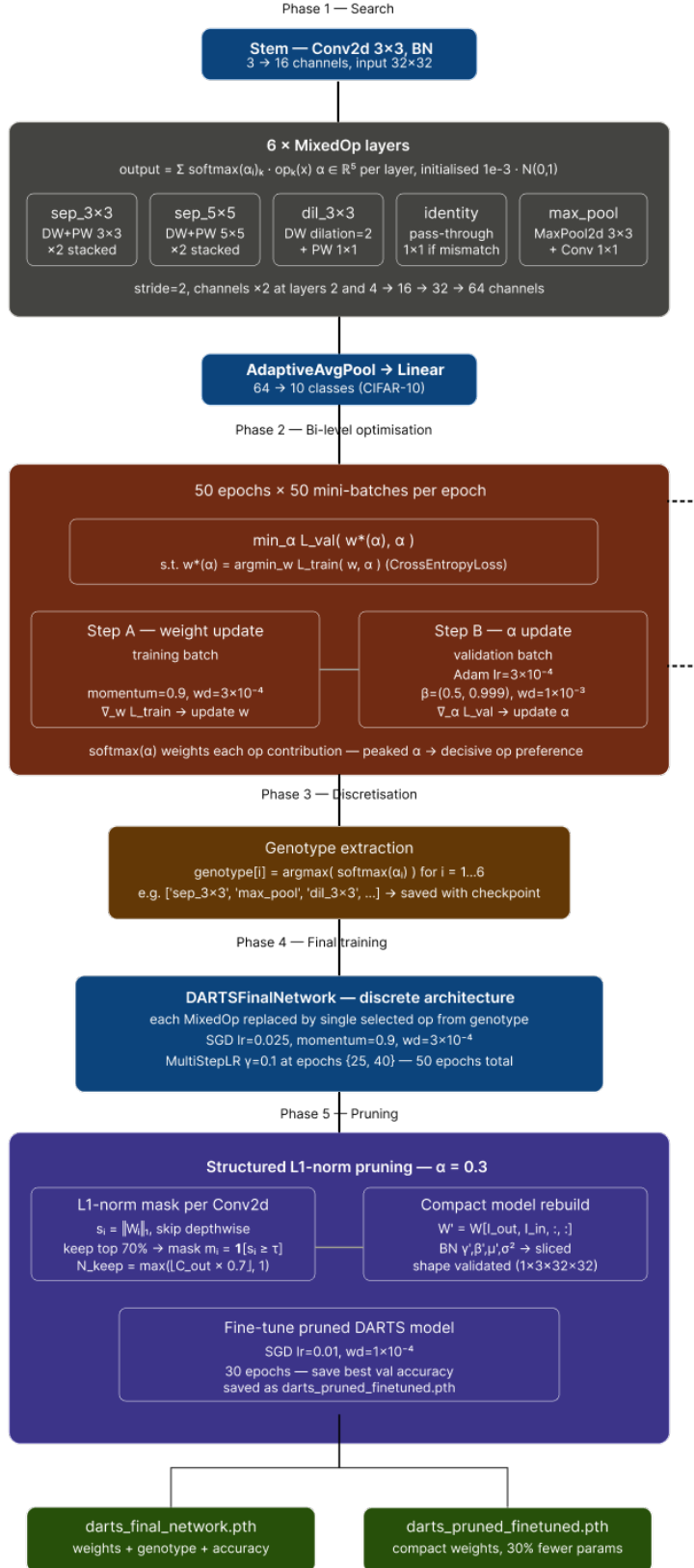


Figure 5.1: DARTS pipeline and pruning with fine-tuning producing

5.1.7 Model Evaluation and Comparison

The four trained model variants Baselines, Pruned Baselines, DARTS Final Network, and DARTS Pruned Network were tested on the CIFAR-10 test set using a shared function called `collect_metrics`. The tested metrics were the test accuracy (Top-1), total number of trainable parameters, number of multiply-accumulate operations (MACs) obtained by computing the number of MACs using the `thop` library with a dummy input tensor of shape $1 \times 3 \times 32 \times 32$, average inference time in milliseconds per image over 20 batches (measured after 3 warmup batches), images per second (throughput), speed-up vs baselines, and latency decrease %. For individual image evaluation, an uploaded image is preprocessed by smart transforming to center-cropped size 32×32 (previous dimension was resized to shorter side of 40), converted into a tensor, and normalized with the values from the CIFAR-10 channel statistics. All models output a softmax probability distribution for the ten possible classes along with the predicted label and confidence for each. A full batch evaluation is also performed on the test set, where the predictions over the whole test set are collected to determine the weighted F1 score using `sklearn.metrics.f1_score` and a class confusion matrix is computed using `sklearn.metrics.confusion_matrix` and displayed as a heatmap. A table showing the results for the classification, confidence, accuracy, and weighted F1 score is presented next to each other for each model.

5.1.8 ONNX Export and Edge Deployment

The four trained model variants are exported into ONNX format. Each model is exported in three formats resulting in twelve files, including FP32 format for the full precision model, and INT8 post-training quantization for the smallest and fast model and prepared(graph optimized) format which applies optimizations like operator fusion and constant folding for balanced speed and accuracy without quantization.

5.1.9 ONNX Runtime Benchmarking

The twelve exported ONNX models are run using ONNX Runtime on the Raspberry Pi with the CPU execution provider, taking a fixed input tensor and running each model 100 times after 10 warmup iterations. The measurements from each of the four met-

rics are saved. This includes the latency in milliseconds of the average, minimum, maximum, and 95th percentile (P95) measurements per inference pass and standard deviation. They also determine the throughput in images per second, speed up relative to FP32 baseline, latency reduction %, and number of parameters in the graph. All the measured metrics are stored in the results file.

5.2 BASELINE MOBILENETV2

5.2.1 Architecture Overview

MobileNetV2 is a lightweight CNN designed for mobile and edge deployment, built around *Inverted Residual Blocks with Linear Bottlenecks* using depthwise separable convolutions. The model is initialised with ImageNet pretrained weights and adapted for CIFAR-10 by replacing the final fully connected layer with a `nn.Linear` mapping to 10 classes. A width multiplier of 0.7 scales all intermediate channel widths, reducing parameter count while retaining the full block topology.

5.2.2 Depthwise Separable Convolution

Depthwise separable convolutions factorise a standard convolution into two stages. A depthwise convolution applies one spatial filter per channel:

$$\hat{G}_{k,l,m}^{\text{DW}} = \sum_{i,j} \hat{K}_{i,j,m} \cdot F_{k+i-1, l+j-1, m} \quad (5.2)$$

followed by a 1×1 pointwise projection across channels. The combined cost reduction relative to a standard convolution is:

$$\text{Reduction} = \frac{1}{N} + \frac{1}{D_K^2} \quad (5.3)$$

For a 3×3 kernel and $N = 32$, this yields an $8\text{--}9\times$ reduction in multiply-accumulate operations.

5.2.3 Inverted Residual Block with Linear Bottleneck

Each block expands a narrow input by factor t , applies a 3×3 depthwise convolution, then projects back to a narrow output. The skip connection links the narrow ends:

$$\mathbf{x}' = \text{PW}_{\text{project}}\left(\text{DW}\left(\text{ReLU6}\left(\text{BN}\left(\text{PW}_{\text{expand}}(\mathbf{x})\right)\right)\right)\right) \quad (5.4)$$

$$\mathbf{y} = \mathbf{x} + \mathbf{x}' \quad (\text{when } \text{stride} = 1, C_{\text{in}} = C_{\text{out}}) \quad (5.5)$$

The final projection has no non-linearity (the *linear bottleneck*), preserving the information manifold in the low-dimensional output space.

5.2.4 Network Structure

Table 5.1: MobileNetV2 architecture configuration for CIFAR-10.

Stage	t	c	n	s
Conv2d 3×3 , BN, ReLU6 (stem)	—	22	1	2
InvertedResidual	1	16	1	1
InvertedResidual	6	24	2	2
InvertedResidual	6	32	3	2
InvertedResidual	6	64	4	2
InvertedResidual	6	96	3	1
InvertedResidual	6	160	3	2
InvertedResidual	6	320	1	1
Conv2d 1×1 , BN, ReLU6 (head)	—	1280	1	1
AdaptiveAvgPool2d, Linear	—	10	1	—

The stem uses 22 channels (standard 32 scaled by 0.7). The final linear layer is the only layer trained from scratch; all others are fine-tuned from ImageNet weights.

5.2.5 Training

The model minimises cross-entropy loss:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^{10} y_{i,c} \cdot \log \left(\frac{e^{\hat{z}_{i,c}}}{\sum_j e^{\hat{z}_{i,j}}} \right) \quad (5.6)$$

using SGD with momentum $\mu = 0.9$ and weight decay $\lambda = 5 \times 10^{-4}$. A `MultiStepLR` scheduler decays the learning rate by $\gamma = 0.1$ at epochs 25 and 40, giving three phases: $\eta = 0.1$ (epochs 0–24), 0.01 (25–39), and 0.001 (40+). Inputs are normalised with CIFAR-10 per-channel statistics ($\mu = (0.4914, 0.4822, 0.4465)$, $\sigma = (0.2023, 0.1994, 0.2010)$), and augmented with random cropping and horizontal flipping during training.

5.3 PRUNED MOBILENETV2

Structured pruning is applied to reduce parameter count and inference cost for deployment on constrained edge hardware. Unlike unstructured pruning, removing entire filters yields a physically smaller, denser network with directly reduced multiply-accumulate operations and memory requirements.

5.3.1 Filter Importance and Mask Construction

Each filter’s importance is its L_1 norm:

$$s_i = \|\mathbf{W}_i\|_1 = \sum_{j,p,q} |W_{i,j,p,q}| \quad (5.7)$$

For pruning ratio $\alpha = 0.3$, the number of filters to retain per layer is:

$$N_{\text{keep}} = \max(\lfloor C_{\text{out}} \cdot (1 - \alpha) \rfloor, 1) \quad (5.8)$$

A binary mask retains filters whose score meets or exceeds the threshold τ (the minimum score among the top- N_{keep} filters):

$$m_i = \mathbf{1}[s_i \geq \tau], \quad \|\mathbf{m}\|_0 = N_{\text{keep}} \quad (5.9)$$

5.3.2 Layer Selection Policy

Depthwise layers (groups > 1) and the final convolutional layer are excluded from pruning to preserve structural integrity. The stem layer and all eligible pointwise (1×1) expansion and projection convolutions within the inverted residual blocks are pruned at the specified ratio.

5.3.3 Compact Model Reconstruction

A compact model is built from scratch using the surviving filter indices $\mathcal{I}_{\text{out}} = \{i : m_i = 1\}$. Weights are sliced as:

$$\mathbf{W}' = \mathbf{W}[\mathcal{I}_{\text{out}}, \mathcal{I}_{\text{in}}, :, :] \quad (5.10)$$

Corresponding BatchNorm2d parameters $(\gamma, \beta, \mu_{\text{run}}, \sigma_{\text{run}}^2)$ are sliced to match. Shape consistency is verified by a dummy forward pass on a $1 \times 3 \times 32 \times 32$ input before fine-tuning.

5.3.4 Fine-tuning

The compact model is fine-tuned for 30 epochs using SGD ($\eta_0 = 0.01$, momentum 0.9, weight decay 5×10^{-4}) with a StepLR schedule ($\gamma = 0.5$ every 15 epochs), giving $\eta = 0.01$ for epochs 0–14 and 0.005 for 15–29. The best validation checkpoint is saved as `prunedfinetuned.pth`.

The full pruning pipeline is illustrated in Figure 5.2: eligible layers are identified, per-filter L_1 scores computed, binary masks constructed retaining the top 70% of filters, and a compact model reconstructed by slicing surviving weights and BatchNorm parameters. A dummy forward pass validates shape consistency before fine-tuning.

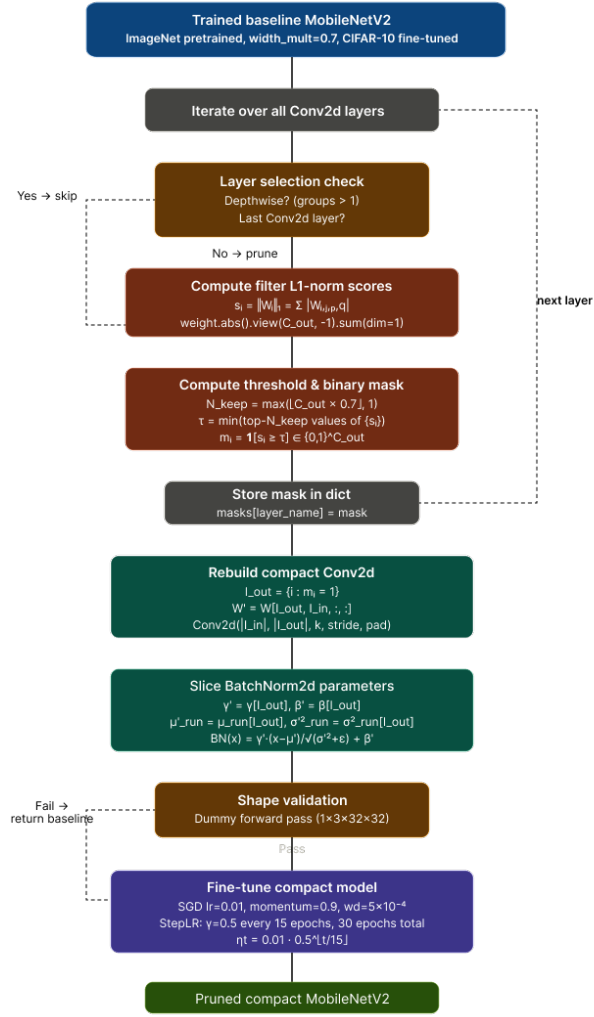


Figure 5.2: Structured pruning pipeline for MobileNetV2.

5.4 BASELINE MOBILENETV3-SMALL

5.4.1 Architecture Overview

MobileNetV3-Small is a hardware-aware CNN combining Neural Architecture Search and human engineering. Over MobileNetV2, it adds *Hard-Swish* activations and *Squeeze-and-Excitation* (SE) blocks for channel-wise recalibration. The model is initialised with IMAGENET1K_V1 weights; only classifier [3] is replaced with a `nn.Linear` mapping to 10 classes.

5.4.2 Hard-Swish and Squeeze-and-Excitation

Hard-Swish replaces ReLU6 in deeper layers using a piecewise linear approximation:

$$\text{HardSwish}(x) = x \cdot \frac{\min(\max(x + 3, 0), 6)}{6} \quad (5.11)$$

SE blocks recalibrate channels by squeezing spatial info via global average pooling, then exciting via two 1×1 convolutions:

$$\mathbf{s} = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \cdot \mathbf{z})), \quad \tilde{U}_c = s_c \cdot U_c \quad (5.12)$$

5.4.3 InvertedResidual Block with SE

Each block expands a narrow input, applies a depthwise convolution, optionally applies SE, then projects back with a skip connection:

$$\mathbf{x}' = \text{PW}_{\text{project}}\left(\text{SE}\left(\text{DW}\left(\text{HardSwish}\left(\text{BN}\left(\text{PW}_{\text{expand}}(\mathbf{x})\right)\right)\right)\right)\right), \quad \mathbf{y} = \mathbf{x} + \mathbf{x}' \text{ (stride} = 1, C_{\text{in}} = C_{\text{out}}) \quad (5.13)$$

5.4.4 Network Structure

Table 5.2: MobileNetV3-Small classifier head adapted for CIFAR-10.

Index	Layer	Description
classifier[0]	Linear	$576 \rightarrow 1024$
classifier[1]	Hardswish	Activation
classifier[2]	Dropout	$p = 0.2$
classifier[3]	Linear	$1024 \rightarrow 10$ (<i>replaced for CIFAR-10</i>)

5.4.5 Training

The model minimises cross-entropy loss using SGD (momentum 0.9, weight decay 5×10^{-4} , $\eta_0 = 0.1$) over 50 epochs. A MultiStepLR scheduler decays η by 0.1 at epochs 25 and 40: $\eta = 0.1 \rightarrow 0.01 \rightarrow 0.001$. Inputs are normalised with CIFAR-10 statistics ($\mu = (0.4914, 0.4822, 0.4465)$, $\sigma = (0.2023, 0.1994, 0.2010)$) and augmented with random cropping and horizontal flipping.

5.5 PRUNED MOBILENETV3-SMALL

Structured pruning targets the *expansion* 1×1 convolution of each InvertedResidual block (where `kernel_size`= (1,1) and `out_channels`>`in_channels`), using ratio $\alpha = 0.5$ — more aggressive than MobileNetV2’s 30% to achieve meaningful reduction given V3’s larger expansion widths. SE blocks are resized during reconstruction to maintain dimensional consistency.

5.5.1 Filter Importance and Mask Construction

Each expansion filter’s importance is its L_1 norm $s_i = \|\mathbf{W}_i\|_1$. The top $N_{\text{keep}} = \max(\lfloor C_{\text{out}} \cdot 0.5 \rfloor, 1)$ filters above threshold τ are retained via boolean mask $b_i = \mathbf{1}[s_i \geq \tau]$.

5.5.2 Block Reconstruction

Each eligible block is deep-copied and rebuilt across four components. The expansion convolution and its BatchNorm are sliced on the output axis:

$$\mathbf{W}'_{\text{expand}} = \mathbf{W}_{\text{expand}}[\mathbf{b}, :, :, :] \quad (5.14)$$

The depthwise convolution is rebuilt with `groups`= C'_{exp} and sliced identically. When an SE block is present, `fc1` and `fc2` are resized to accept and emit C'_{exp} channels while preserving the internal reduction dimension r_{original} :

$$\text{fc1} : C'_{\text{exp}} \rightarrow r_{\text{original}}, \quad \text{fc2} : r_{\text{original}} \rightarrow C'_{\text{exp}} \quad (5.15)$$

The projection convolution is updated only on its input axis, leaving output channels unchanged:

$$\mathbf{W}'_{\text{project}} = \mathbf{W}_{\text{project}}[:, \mathbf{b}, :, :] \quad (5.16)$$

5.5.3 Fine-tuning

The compact model is verified by parameter reduction check $R = 1 - |\theta_{\text{pruned}}|/|\theta_{\text{baseline}}|$ (targeting 35–40%), then fine-tuned for 50 epochs using Adam with CosineAnnealingLR and label smoothing $\varepsilon = 0.1$. The full pipeline is shown in Figure 5.3.

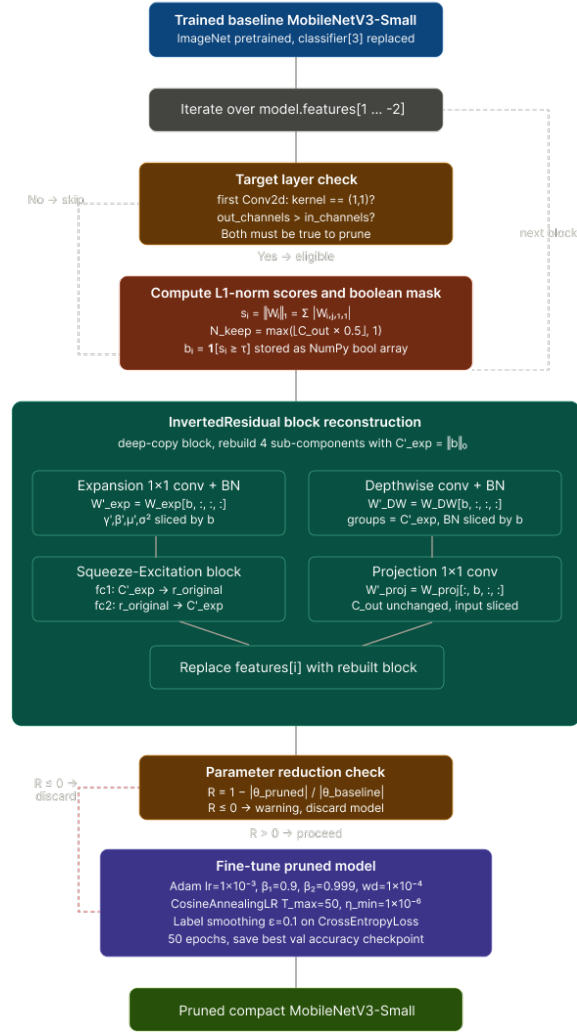


Figure 5.3: Structured pruning pipeline for MobileNetV3-Small, showing the stage-by-stage process.

5.6 BASELINE EFFICIENTNET

5.6.1 Architecture Overview

EfficientNet is the base model of the EfficientNet family, scaling depth, width, and resolution jointly via a compound coefficient derived through Neural Architecture Search. Its core building block is the *MBConv* layer — a mobile inverted bottleneck with an integrated SE module. The model is initialised with IMAGENET1K_V1 weights; only `classifier[1]` is replaced with a `nn.Linear` mapping from 1280 features to 10 classes.

5.6.2 Swish Activation and MBConv Block

EfficientNet uses Swish throughout its MBConv blocks:

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5.17)$$

Each MBConv block expands, applies a depthwise convolution with SE recalibration, then projects back with a skip connection:

$$\mathbf{x}' = \text{PW}_{\text{project}}\left(\text{SE}\left(\text{DW}\left(\text{Swish}\left(\text{BN}\left(\text{PW}_{\text{expand}}(\mathbf{x})\right)\right)\right)\right)\right), \quad \mathbf{y} = \mathbf{x} + \mathbf{x}' \text{ (stride} = 1, C_{\text{in}} = C_{\text{out}}) \quad (5.18)$$

For MBConv1 blocks ($t = 1$), the pointwise expansion is omitted.

5.6.3 Network Structure

Table 5.3: EfficientNet classifier head adapted for CIFAR-10.

Index	Layer	Description
classifier[0]	Dropout	$p = 0.2$
classifier[1]	Linear	$1280 \rightarrow 10$ (<i>replaced for CIFAR-10</i>)

5.6.4 Training

The model minimises cross-entropy loss using SGD (momentum 0.9, weight decay 5×10^{-4} , $\eta_0 = 0.1$) over 50 epochs. A MultiStepLR scheduler decays η by 0.1 at epochs 25 and 40: $\eta = 0.1 \rightarrow 0.01 \rightarrow 0.001$. Inputs are normalised with CIFAR-10 statistics ($\mu = (0.4914, 0.4822, 0.4465)$, $\sigma = (0.2023, 0.1994, 0.2010)$) and augmented with random cropping and horizontal flipping.

5.7 PRUNED EFFICIENTNET

Structured pruning targets the expansion 1×1 convolution of eligible MBConv blocks using L_1 -norm filter ranking with $\alpha = 0.3$. Three differences from MobileNet pruning apply: MBConv1 blocks (no expansion convolution) are skipped; a consistency check $C_{\text{out}}^{\text{exp}} = C_{\text{in}}^{\text{proj}}$ must hold; and the SE module is resized to match the pruned expansion width.

5.7.1 Block Analysis, Filter Importance, and Mask Construction

Each block is structurally analysed to identify the expansion convolution (the 1×1 Conv2d immediately preceding the depthwise), depthwise, and projection convolutions. Blocks failing identification or the consistency check are skipped. For eligible blocks, filter importance $s_i = \|\mathbf{W}_i\|_1$ is computed and the top $N_{\text{keep}} = \max(\lfloor C_{\text{out}} \cdot 0.7 \rfloor, 1)$ filters are retained via mask $m_i = \mathbf{1}[s_i \geq \tau]$.

5.7.2 MBConv Block Reconstruction

Each eligible block is deep-copied and rebuilt with reduced width $C'_{\text{exp}} = \|\mathbf{m}\|_0$ across five components. The expansion convolution and both BatchNorm layers are sliced on the output axis:

$$\mathbf{W}'_{\text{expand}} = \mathbf{W}_{\text{expand}}[\mathbf{m}, :, :, :], \quad \gamma' = \gamma[\mathbf{m}], \beta' = \beta[\mathbf{m}], \mu'_{\text{run}} = \mu_{\text{run}}[\mathbf{m}], \sigma'^2_{\text{run}} = \sigma^2_{\text{run}}[\mathbf{m}] \quad (5.19)$$

The depthwise convolution is rebuilt with $\text{groups} = C'_{\text{exp}}$ and sliced identically. The SE block is resized to accept and emit C'_{exp} channels while preserving the internal reduction dimension:

$$\text{SE_reduce} : C'_{\text{exp}} \rightarrow r_{\text{original}}, \quad \text{SE_expand} : r_{\text{original}} \rightarrow C'_{\text{exp}} \quad (5.20)$$

The projection convolution is updated only on its input axis:

$$\mathbf{W}'_{\text{project}} = \mathbf{W}_{\text{project}}[:, \mathbf{m}, :, :] \quad (5.21)$$

5.7.3 Fine-tuning

The compact model is verified by parameter reduction ratio $R = 1 - |\theta_{\text{pruned}}|/|\theta_{\text{baseline}}|$, then fine-tuned for 30 epochs using SGD ($\eta_0 = 0.01$, momentum 0.9, weight decay 5×10^{-4}) with StepLR ($\gamma = 0.5$ every 15 epochs), giving $\eta = 0.01 \rightarrow 0.005$. The best checkpoint is saved as `pruned_efficientnet_finetuned.pth`. The full pipeline is shown in Figure 5.4.

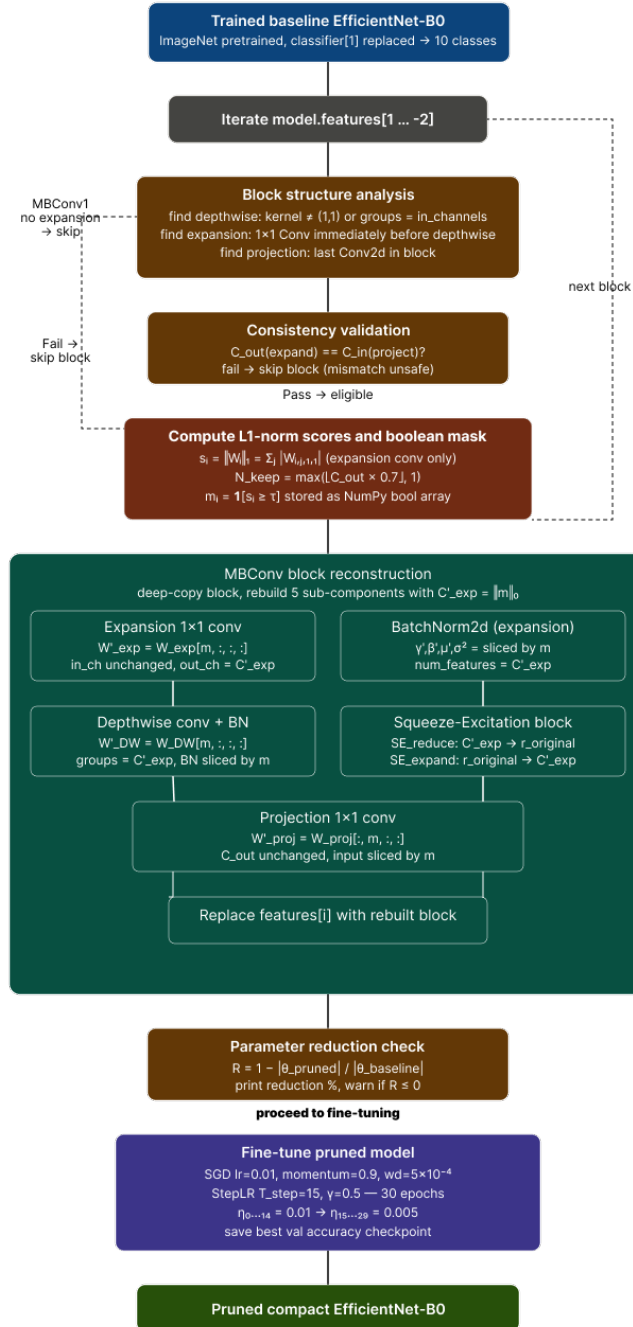


Figure 5.4: Structured pruning pipeline for EfficientNet , showing the stage-by-stage process from trained baseline.

5.8 TESTING

5.8.1 Overview

Tests are performed at two separate levels. The first level tests all the versions of the trained models in CIFAR-10 test set under well- controlled software conditions, where their accuracy, efficiency, and class discrimination ability are assessed. At the second level, all twelve ONNX models are deployed in Raspberry Pi 4, which assesses the practical latency, throughput and compression benefit gained from the process quantization and graph optimization in real-life conditions.

5.8.2 Test Environment

Software Training Environment

All training from scratch, pruning, fine-tuning, and software-based evaluation takes place in Google Colab using one NVIDIA GPU (either T4 or A100, whichever is available). The software infrastructure includes Python 3.10, PyTorch 2.x, torchvision, NumPy 1.24, scikit-learn for F1 and confusion matrix calculations, thop for multiply-accumulate operation counting, and seaborn for plotting confusion matrices. The experiments use a consistent random seed value of 42 within PyTorch, NumPy, and the Python built-in module at the beginning of each experiment to guarantee reproducibility of weight initialisation, data shuffling, and dropout masks.

Edge Deployment Environment

Benchmarking for dge is carried out on a Raspberry Pi 4 Model B device having 4 GB RAM and running the Raspberry Pi OS, 64-bit. Benchmarking for dge is done solely through the CPU, using ONNX Runtime 1.16 with four threads in the CPU Execution Provider. No graphics processor or any other type of hardware accelerator is used for inference; hence, it represents real-world scenarios of low-cost edge devices. All ONNX files are benchmarked from a zip file containing all the 12 files to the Raspberry Pi device through a Python script. Input for all the models is a tensor of size (1,3,32,32).

5.8.3 Test Cases

Test Case 1 : Baseline Model Accuracy

For each of the three baseline architectures (MobileNetV2, MobileNetV3-Small, and EfficientNet), an evaluation against the complete 10,000-image CIFAR-10 test set is conducted. First, the model is placed into inference mode using `model.eval()`, with all gradient calculations turned off via `torch.no_grad()`. The predicted label for each image is found by selecting the maximum logit index, and accuracy is determined from the percentage of correctly classified samples.

Expected outcome: All three baselines achieve competitive Top-1 accuracy on CIFAR-10, reflecting the benefit of ImageNet pretraining combined with domain-specific fine-tuning.

Test Case 2 : Pruned Model Accuracy Recovery

For each pruned and refined model, the performance on CIFAR-10 is compared against that of the baseline model under similar circumstances. This performance difference is captured in the form of $\Delta\text{Acc} = \text{Acc}_{\text{Baseline}} - \text{Acc}_{\text{Pruned}}$. A small performance difference implies that the fine-tuning has been successful in regaining lost representation capabilities.

Expected outcome: Each pruned model recovers to within 1–2% of its baseline accuracy, demonstrating that structured pruning followed by fine-tuning preserves classification performance while reducing model size.

Test Case 3 : Parameter and MAC Reduction

For each pair of models, one from baseline and another from pruned, the total number of trainable parameters and MAC operations is calculated before and after pruning. Both $\text{CR}_{\text{params}}$ and CR_{MACs} are determined using:

$$\text{CR}_{\text{params}} = \frac{|\theta_{\text{baseline}}|}{|\theta_{\text{pruned}}|}, \quad \text{CR}_{\text{MACs}} = \frac{\text{MACs}_{\text{baseline}}}{\text{MACs}_{\text{pruned}}} \quad (5.22)$$

Expected outcome: All three pruned models demonstrate measurable reductions in both parameter count and MACs relative to their baselines, with MobileNetV3-Small

showing the largest reduction owing to its higher pruning ratio of $\alpha = 0.5$.

Test Case 4 : Inference Latency and Throughput

Per-image inference latency is measured for all six PyTorch model variants on the GPU training environment. Each model is run for 3 warm-up batches to stabilise CUDA kernels and memory allocations, followed by 20 timed measurement batches. The mean per-image latency and images-per-second throughput are recorded. Speedup over the corresponding baseline is computed as:

$$\text{Speedup} = \frac{\bar{t}_{\text{baseline}}}{\bar{t}_{\text{model}}} \quad (5.23)$$

and latency reduction as:

$$\text{Latency Reduction (\%)} = \left(1 - \frac{\bar{t}_{\text{model}}}{\bar{t}_{\text{baseline}}} \right) \times 100 \quad (5.24)$$

Expected outcome: Pruned models exhibit lower per-image latency and higher throughput than their baselines, with the degree of improvement proportional to the parameter reduction achieved.

Test Case 5 : Per-Class Performance and Confusion Matrix

For each of the six model variants, predictions are collected across the entire CIFAR-10 test set and used to compute the weighted F1 score and a per-class confusion matrix. The weighted F1 score accounts for class imbalance by weighting the per-class F1 by the number of samples in each class:

$$F_1^{\text{weighted}} = \sum_{c=1}^{10} \frac{n_c}{N_{\text{test}}} \cdot \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \quad (5.25)$$

where P_c and R_c are the precision and recall for class c , and n_c is the number of test samples in class c . The confusion matrix is visualised as a normalised heatmap to identify which class pairs are most frequently confused, providing qualitative insight into the failure modes of each model and whether pruning introduces new misclassification patterns absent in the baseline.

Expected outcome: Weighted F1 scores closely track Top-1 accuracy across all

models. Confusion matrices reveal that visually similar classes (e.g. cat–dog, automobile–truck) account for the majority of misclassifications consistently across all variants.

Test Case 6 : Per-Image Visual Inference

A set of test images from the CIFAR-10 dataset is fed one at a time into the six variants of the network using the smart inference preprocessing pipeline, where the image is first resized such that its shorter dimension is 40 pixels, then center cropped to 32×32 , followed by conversion to a tensor and normalisation using the mean and standard deviation of the channels in the CIFAR-10 dataset. The output class label and softmax probability score are obtained for each variant.

Expected outcome: All six models agree on the predicted class for the majority of unambiguous test images. Cases where pruned models disagree with their baselines are expected to correspond to lower confidence scores, indicating greater prediction uncertainty rather than systematic misclassification.

Test Case 7 : ONNX Export Validation

Each of the 12 exported ONNX models is validated immediately after export using `onnx.checker.check_model`, which verifies node connectivity, operator type compliance, and graph structural integrity. An ONNX Runtime inference session is then instantiated for each model using the CPU execution provider, and a test forward pass is executed with a fixed input of shape (1,3,32,32). The output tensor shape is checked to confirm it matches the expected (1,10) logit vector.

Expected outcome: All 12 ONNX models pass structural validation and produce correctly shaped outputs, confirming that the export procedure preserves the model's computational graph and that all three export formats (FP32, INT8, and Prep) are loadable and executable on the target runtime.

Test Case 8 : Edge Latency and Quantisation Impact

All 12 ONNX models are benchmarked on the Raspberry Pi using 10 warmup iterations followed by 100 timed inference runs per model. For each model the following statistics are recorded: average latency, minimum latency, maximum latency, 95th percentile latency (P95), standard deviation of latency, throughput in images per second, speedup

relative to the corresponding FP32 baseline, latency reduction percentage, model file size in megabytes, and parameter count extracted from the ONNX graph. The INT8 quantised variants are specifically evaluated for the accuracy-latency tradeoff, as post-training quantisation converts 32-bit floating-point weights to 8-bit integers, reducing model file size by approximately $4\times$ and exploiting integer arithmetic units for faster CPU inference. The Prep (graph-optimised) variants are evaluated to isolate the contribution of ONNX-level operator fusion and constant folding independent of numerical precision changes.

5.8.4 Evaluation Metrics

Top-1 Classification Accuracy

Top-1 accuracy is the primary classification metric, defined as the fraction of test samples for which the predicted class matches the ground-truth label:

$$\text{Accuracy} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbf{1} \left[\arg \max_c \hat{z}_{i,c} = y_i \right] \quad (5.26)$$

It is reported as a percentage and computed over all 10,000 CIFAR-10 test samples.

Multiply-Accumulate Operations

MACs measure the total number of multiply-accumulate operations required for a single forward pass, providing a hardware-independent proxy for computational cost. They are estimated using the `thop` library with a dummy input of shape $1 \times 3 \times 32 \times 32$. A lower MAC count indicates a more computationally efficient model.

Inference Latency

Per-image inference latency $\bar{t}$ (ms) is the mean time required to process a single image through the full forward pass, averaged over 20 measurement batches following 3 warm-up batches on the GPU, and over 100 runs following 10 warm-up iterations on the Raspberry Pi. It is the most direct measure of deployment suitability for real-time applications.

Throughput

Throughput is defined as the number of images processed per second:

$$\text{FPS} = \frac{1000}{\bar{t} \text{ (ms)}} \quad (5.27)$$

Higher throughput indicates greater processing capacity under sustained inference workloads.

P95 Latency

The 95th percentile latency (P95) is recorded during edge benchmarking as a measure of worst-case real-world performance. While average latency reflects typical behaviour, P95 captures the effect of operating system scheduling jitter and thermal throttling that occasionally inflate individual inference times on embedded hardware.

Model File Size

The model file size in megabytes is recorded for each of the 12 ONNX variants. It reflects the on-disk storage requirement and is directly relevant to deployment on memory-constrained edge devices.

Speedup and Latency Reduction

Speedup is computed as the ratio of the baseline inference time to the model inference time (Equation 5.23). A speedup greater than 1.0 indicates a faster model. Latency reduction percentage expresses the same improvement as a proportion (Equation 5.24). On the edge benchmark, both metrics are computed relative to the corresponding FP32 baseline model rather than the PyTorch baseline, isolating the benefit of compression and quantisation in the deployment environment.

Weighted F1 Score

The weighted F1 score (Equation 5.25) provides a more informative summary of classification performance than accuracy alone when class-level precision and recall differ. It penalises models that achieve high accuracy by over-predicting frequent classes at the expense of recall on rarer ones.

CHAPTER 6

PROJECT IMPLEMENTATION

The present chapter provides an overview of the network compression pipeline, including its definition, structure, components, and requirements for successful implementation. Both software-related aspects and hardware prerequisites will be considered, along with the necessary settings that ensure convenient operation. The complete process will be implemented using Python, PyTorch, and the torchvision framework. The computations will be carried out in Google Colab, which will allow performing tasks with the help of the GPU. We have considered MobileNetV2, MobileNetV3, EfficientNet, and DARTS in their initial and pruned forms. The mentioned neural networks have been transformed into the ONNX format and evaluated using the Raspberry Pi 4.

6.1 DEVELOPMENT ENVIRONMENT

6.1.1 Hardware

In the meantime, training and pruning processes have been accomplished on Google Colab Pro using a workstation with an NVIDIA T4 GPU (16 GB VRAM and 12 GB system RAM). To assess the edge inference latency, we have employed a Raspberry Pi 4 Model B with 4 GB of LPDDR4 RAM and a 64-bit operating system (Raspberry Pi OS). At the same time, no accelerators have been applied, namely NPU and GPU.

6.1.2 Software Stack

The following software versions were used throughout the project:

Table 6.1: Software environment used for training, pruning and deployment.

Package	Version
Python	3.10
PyTorch	2.x (CUDA 11.8)
torchvision	0.15.x
ONNX	1.14
ONNX Runtime	1.16
thop	0.1.x
NumPy	1.24
scikit-learn	1.3
seaborn	0.12
matplotlib	3.7

6.1.3 Reproducibility

Before any experiments, the seed value of the random number generator was set to 42. For consistent outcomes while training on GPUs, `torch.backends.cudnn.deterministic` is set to True, but `torch.backends.cudnn.The benchmark` is set to false. Because of this, weight initialisations, data shuffles, and dropouts are the same every time the program runs.

6.2 DATASET PREPARATION IMPLEMENTATION

The CIFAR-10 data set is downloaded automatically using the `torchvision.datasets.CIFAR10` class with `download=True` attribute. The CIFAR-10 training data set with 50K samples was loaded via the transformation pipeline consisting of `RandomCrop(32, padding=4)`, `RandomHorizontalFlip()`, and normalization with the mean values equal to (0.4914, 0.4822, 0.4465) and the standard deviation values equal to (0.2023, 0.1994, 0.2010) per each color channel. The CIFAR-10 testing data set with 10K samples was loaded only with normalization transformation.

6.3 BASELINE MODEL IMPLEMENTATION

6.3.1 MobileNetV2

Baseline MobileNetV2 model was created using torchvision.models.mobilenet_v2 using pre-trained weights on ImageNet and the value of the width multiplier set to 0.7. In addition, the final layer of the model was substituted with nn.Linear(1280, 10) in order to modify it to recognize 10 categories of CIFAR-10 images. Training of the model was done using an optimizer with parameters being set as follows: the number of epochs = 50, learning rate = 0.1, momentum = 0.9, weight decay = 5×10^{-4} .

The following code snippet demonstrates the baseline MobileNetV2 model instantiation and classifier adaptation:

```
def get_mobilenet_v2(num_classes=10, pretrained=True):
    model = models.mobilenet_v2(
        weights="IMAGENET1K_V1" if pretrained else None
    )
    in_features = model.classifier[1].in_features
    model.classifier[1] = nn.Linear(in_features, num_classes)
    return model
```

Listing 6.1: Baseline MobileNetV2 model definition and classifier adaptation.

6.3.2 MobileNetV3-Small

For the MobileNetV3-Small baseline implementation, we used torchvision.models.mobilenet_v3 with MobileNet V3 Small Weights.IMAGENET1K_V1. We modified the classifier layer at the third index (model.classifier[3]) by replacing it with nn.Linear(1024, 10). We used the same SGD-based training configuration and MultiStepLR schedule as we did in MobileNetV2. The best checkpoint for MobileNetV3-Small was saved as baseline mobilenetv3.pth.

6.3.3 EfficientNet

We instantiated the EfficientNet baseline using torchvision.models.efficientnet with weights="IMAGENET1K_V1". The classifier layer of the model (model.classifier[1]) was modified to use nn.Linear(1280, 10) for the 10-class CIFAR-10 classification task. Training was done using the same SGD-based training method and MultiStepLR schedule as the other 54 baselines. The best checkpoint was saved as baseline efficientnet.pth

6.4 STRUCTURED PRUNING IMPLEMENTATION

All three of the baseline models were pruned using structured pruning. Structured pruning was conducted using a three-step process: (1) filter importance scoring based on L1-norm ranking; (2) rebuilding the model physically with reduced channel dimensions; and (3) fine-tuning of the resulting pruned model to recover accuracy lost during the pruning process.

6.4.1 Filter Importance Scoring

Filter importance scoring constitutes the basis of structured pruning. For each convolutional layer, the L1-norm of the filter weights is determined. The L1-norm values are then used to assign relative importance to each of the filters.

The following code snippet demonstrates the L1-norm filter importance scoring and binary mask construction process:

```
def extract_pruning_masks(model, prune_amount=0.3,
                          skip_depthwise=True,
                          skip_first_last=True):
    masks = {}
    layer_names = [name for name, m in model.named_modules()
                   if isinstance(m, nn.Conv2d)]
    last_layer = layer_names[-1]

    for name, module in model.named_modules():
        if not isinstance(module, nn.Conv2d):
            continue
        if skip_first_last and name == last_layer:
            continue
        if skip_depthwise and module.groups > 1:
            continue

        weight = module.weight.data.abs().clone()
        l1_norms = weight.view(weight.size(0), -1).sum(dim=1)
        num_keep = max(int(weight.size(0) * (1 - prune_amount)), 1)
        threshold = torch.topk(l1_norms, num_keep).values.min()
        mask = (l1_norms >= threshold).cpu().numpy()
        masks[name] = mask

    return masks
```

Listing 6.2: L1-norm filter importance scoring and pruning mask construction.

The `extract_pruning_masks` function skips depthwise convolutions (groups > 1) and the final classification layer to preserve input and output dimensionality constraints. A pruning ratio of 30% removes the bottom 30% of filters ranked by L1-norm, retaining the top 70%.

6.4.2 Model Reconstruction Utilities

Once the chosen filters have been decided, the model can be retrained with fewer filters. What this involves is cutting down on the number of filters by reducing the weight matrices and ensuring that all layers dependent on them (batch normalization layers and subsequent convolutional layers included) match this reduced figure.

The following utilities demonstrate the reconstruction of convolutional and batch normalisation layers with reduced channel counts:

```
def rebuild_conv_layer(conv, out_mask, in_mask=None):
    out_indices = np.where(out_mask)[0]
    new_out     = len(out_indices)
    new_in      = conv.in_channels if in_mask is None \
                  else len(np.where(in_mask)[0])
    in_indices  = None if in_mask is None \
                  else np.where(in_mask)[0]

    new_conv = nn.Conv2d(new_in, new_out,
                        conv.kernel_size, conv.stride,
                        conv.padding, bias=(conv.bias is not None))

    if in_indices is not None:
        new_conv.weight.data = \
            conv.weight.data[out_indices][:, in_indices].clone()
    else:
        new_conv.weight.data = \
            conv.weight.data[out_indices].clone()

    if conv.bias is not None:
        new_conv.bias.data = conv.bias.data[out_indices].clone()

    return new_conv

def rebuild_batchnorm_layer(bn, mask):
    indices = np.where(mask)[0]
    new_bn  = nn.BatchNorm2d(len(indices))
    new_bn.weight.data = bn.weight.data[indices].clone()
    new_bn.bias.data   = bn.bias.data[indices].clone()
    new_bn.running_mean = bn.running_mean[indices].clone()
```

```

new_bn.running_var = bn.running_var[indices].clone()
return new_bn

```

Listing 6.3: Compact weight tensor reconstruction for pruned convolution and batch normalisation layers.

These utilities handle both output channel pruning (removing entire filters) and input channel pruning (removing connections to pruned filters in the previous layer), ensuring dimensional consistency throughout the network.

6.4.3 MobileNetV2 Pruning

The trained MobileNetV2 baseline was loaded from `baseline_mobilenetv2.pth`. The `extract_pruning_masks` function iterated over all `nn.Conv2d` layers, skipping depthwise layers (`groups > 1`) and the final convolutional layer. For each eligible layer, the L1-norm of each filter was computed and the bottom 30% of filters were identified for removal. A binary boolean mask was stored per layer.

The `build_conservative_compact_mobilenetv2` function physically reconstructed a smaller model by slicing the weight tensors to the surviving filter indices and rebuilding the corresponding BatchNorm2d layers. A dummy forward pass on a $1 \times 3 \times 32 \times 32$ input validated the shape consistency of the compact model.

Fine-tuning was performed for 30 epochs using SGD with learning rate 0.01 and a StepLR scheduler decaying by 0.5 every 15 epochs. The best checkpoint was saved as `pruned_mobilenetv2_finetuned.pth`.

6.4.4 MobileNetV3-Small Pruning

MobileNetV3-Small’s inverted residual blocks present unique pruning challenges due to the presence of Squeeze-and-Excitation (SE) modules that must be resized alongside the expansion convolutions. The `extract_mobilenetv3_masks` function targeted only the expansion 1×1 convolution in each InvertedResidual block, identified by the conditions `kernel_size == (1,1)` and `out_channels > in_channels`. The stem (`features[0]`) and head (`features[-1]`) were skipped by index-based guards. A pruning ratio of 0.5 was applied, retaining the top 50% of filters.

The following code demonstrates the MobileNetV3-Small block reconstruction procedure, which updates the expansion convolution, depthwise convolution, SE module,

and projection convolution to maintain dimensional consistency:

```
def _rebuild_block_children(container, mask, children_list):
    out_ch = int(mask.sum())
    conv_norm_acts = [c for c in children_list
                       if isinstance(c, (Conv2dNormActivation,
                                         nn.Sequential))]
    se_blocks = [c for c in children_list
                 if isinstance(c, SqueezeExcitation)]

    # Expansion 1x1 conv + BN
    expand = conv_norm_acts[0]
    old_c0, old_b0 = expand[0], expand[1]
    new_c0 = nn.Conv2d(old_c0.in_channels, out_ch, 1, bias=False)
    new_c0.weight.data = old_c0.weight.data[mask].clone()
    new_b0 = nn.BatchNorm2d(out_ch)
    new_b0.weight.data = old_b0.weight.data[mask].clone()
    new_b0.bias.data = old_b0.bias.data[mask].clone()
    new_b0.running_mean = old_b0.running_mean[mask].clone()
    new_b0.running_var = old_b0.running_var[mask].clone()
    expand[0], expand[1] = new_c0, new_b0

    # Depthwise conv + BN
    dw = conv_norm_acts[1]
    old_c1, old_b1 = dw[0], dw[1]
    new_c1 = nn.Conv2d(out_ch, out_ch, old_c1.kernel_size,
                       old_c1.stride, old_c1.padding,
                       groups=out_ch, bias=False)
    new_c1.weight.data = old_c1.weight.data[mask].clone()
    new_b1 = nn.BatchNorm2d(out_ch)
    new_b1.weight.data = old_b1.weight.data[mask].clone()
    new_b1.bias.data = old_b1.bias.data[mask].clone()
    new_b1.running_mean = old_b1.running_mean[mask].clone()
    new_b1.running_var = old_b1.running_var[mask].clone()
    dw[0], dw[1] = new_c1, new_b1

    # Squeeze-Excitation: resize fc1 input and fc2 output
    if se_blocks:
        se = se_blocks[0]
        se.fc1 = nn.Conv2d(out_ch, se.fc1.out_channels, 1)
        se.fc2 = nn.Conv2d(se.fc2.in_channels, out_ch, 1)

    # Projection 1x1: slice input channels only
    proj = conv_norm_acts[2]
    old_c3 = proj[0]
    new_c3 = nn.Conv2d(out_ch, old_c3.out_channels, 1, bias=False)
    new_c3.weight.data = old_c3.weight.data[:, mask, :, :].clone()
    proj[0] = new_c3
```

Listing 6.4: MobileNetV3-Small block reconstruction with Squeeze-and-Excitation re-sizing.

The `build_pruned_mobilenetv3` function applied this reconstruction to each eligible block, updating the expansion convolution, depthwise convolution, both corresponding BatchNorm2d layers, and the Squeeze-and-Excitation `fc1` and `fc2` convolutions to match the reduced expansion width.

Fine-tuning used Adam with learning rate 1×10^{-3} , CosineAnnealingLR over 50 epochs, and label smoothing $\varepsilon = 0.1$ to prevent overfitting. The best checkpoint was saved as `pruned_mobilenetv3_finetuned.pth`.

6.4.5 EfficientNet Pruning

EfficientNet’s Mobile Inverted Bottleneck Convolution (MBConv) blocks require auto-detection of expansion, depthwise, and projection convolutions due to architectural variations between MBConv1 (no expansion) and MBConv6 (6× expansion) blocks. The `analyze_block_structure` function traversed each MBConv block to auto-detect the expansion, depthwise, and projection convolutions by kernel size and position. MBConv1 blocks where no expansion convolution was found were skipped. A consistency check confirmed that $C_{\text{out}}^{\text{exp}} = C_{\text{in}}^{\text{proj}}$ before pruning proceeded.

L1-norm masks were computed at $\alpha = 0.3$ over the expansion convolution filters. The `rebuild_mbconv_block_robust` function reconstructed each eligible block, updating the expansion convolution, depthwise convolution, both BatchNorm2d layers, the SE reduce and expand convolutions, and the projection convolution.

Fine-tuning used SGD with learning rate 0.01 and a StepLR scheduler decaying by 0.5 every 15 epochs over 30 epochs.

6.5 DARTS IMPLEMENTATION

The Differentiable Architecture Search (DARTS) implementation followed a two-stage process: (1) architecture search using bi-level optimisation to learn both network weights and architecture parameters, and (2) discretisation and training of the final architecture from scratch.

6.5.1 Search Phase

The DARTSSearchNetwork was constructed with 16 initial channels and 6 layers. Each layer was implemented as a MixedOp holding five candidate operations: sep_3x3 (3×3 separable convolution), sep_5x5 (5×5 separable convolution), dil_3x3 (3×3 dilated convolution), identity (skip connection), and max_pool (3×3 max pooling). Architecture parameters α were initialised as $10^{-3} \cdot \mathcal{N}(0, 1)$ per layer to provide small random perturbations around zero.

The bi-level optimisation ran for 50 epochs of 50 mini-batches each. Network weights were updated on training batches using SGD with learning rate 0.025, momentum 0.9, and weight decay 3×10^{-4} . Architecture parameters were updated on validation batches using Adam with learning rate 3×10^{-4} , $\beta = (0.5, 0.999)$, and weight decay 1×10^{-3} .

The following code demonstrates the MixedOp forward pass and genotype extraction mechanism:

```
class MixedOp(nn.Module):
    def __init__(self, C_in, C_out, stride):
        super().__init__()
        self.ops = nn.ModuleList([
            op(C_in, C_out, stride)
            for op in get_operations(C_in, C_out, stride).values()
        ])

    def forward(self, x, weights):
        return sum(w * op(x)
                    for w, op in zip(weights, self.ops))

class DARTSSearchNetwork(nn.Module):
    def forward(self, x):
        x = self.stem(x)
        for i, layer in enumerate(self.layers):
            weights = F.softmax(self.alphas[i], dim=0)
            x = layer(x, weights)
        x = self.global_pooling(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

    def genotype(self):
        return [
            list(get_operations(1, 1, 1).keys())[
                alpha.argmax().item()
            ]
        ]
```



```

    ]
    for alpha in self.alphas
]

```

Listing 6.5: DARTS MixedOp forward pass with weighted operation combination and genotype extraction.

During the search phase, each MixedOp computes a weighted sum of all candidate operations, where the weights are derived from softmax-applied architecture parameters. This differentiable formulation allows gradient-based optimisation of the architecture topology.

6.5.2 Discretisation and Final Network Training

After the search, the genotype was extracted by taking $\arg \max(\text{softmax}(\alpha_i))$ at each of the six layers, producing a list of six operation names. This discretisation step selects the single highest-weighted operation at each layer, converting the continuous architecture representation into a discrete network topology.

A clean DARTSFinalNetwork was constructed from the genotype and trained from scratch for 50 epochs using SGD with learning rate 0.025, momentum 0.9, weight decay 3×10^{-4} , and a MultiStepLR scheduler decaying by 0.1 at epochs 25 and 40. Training from scratch ensures that the final network learns optimal weights for the discovered architecture without contamination from the search phase weights. The best checkpoint was saved as `darts_final_network.pth` alongside the genotype.

6.5.3 DARTS Pruning

Structured L1-norm pruning at $\alpha = 0.3$ was applied to the trained DARTS Final Network following the same filter ranking and mask construction procedure used for the CNN baselines. Depthwise convolutions were skipped to preserve separable convolution structure. The compact DARTS model was fine-tuned for 30 epochs using SGD with learning rate 0.01 and weight decay 1×10^{-4} , and saved as `darts_pruned_finetuned.pth`.

6.6 ONNX EXPORT IMPLEMENTATION

All six trained model variants (three baselines and three pruned models) were exported to ONNX format in multiple configurations to evaluate the impact of precision reduc-

tion and graph optimisation on deployment performance. The export process generated three versions of each model: FP32 (full precision), Prep (graph-optimised), and INT8 (quantised), producing 18 ONNX files in total.

The FP32 export used `torch.onnx.export` with opset version 17 and a static input shape of $(1, 3, 32, 32)$. This configuration ensures compatibility with ONNX Runtime 1.16 and enables efficient batch-size-1 inference for edge deployment scenarios.

The Prep (graph-optimised) export applied ONNX Runtime's `quant_pre_process` transformation for operator fusion and constant folding without quantisation. This step merges consecutive operations (e.g., `Conv`→`BatchNorm`→`ReLU`) into single fused kernels and pre-computes constant expressions to reduce runtime overhead.

The INT8 export applied post-training dynamic quantisation using `onnxruntime` with `QuantType.QUInt8`, reducing model file size by approximately $4\times$ compared to FP32. Dynamic quantisation performs weight quantisation offline while keeping activations in floating-point during inference, balancing accuracy preservation with storage reduction.

The following code demonstrates the complete ONNX export and quantisation pipeline:

```
def export_and_quantize_onnx(model, base_filename):
    model.eval()
    model.cpu()
    dummy = torch.randn(1, 3, 32, 32)
    fp32_path = f"{ONNX_DIR}/{base_filename}_fp32.onnx"
    int8_path = f"{ONNX_DIR}/{base_filename}_int8.onnx"

    # FP32 export
    torch.onnx.export(model, dummy, fp32_path,
                      opset_version=17,
                      do_constant_folding=True,
                      input_names=['input'],
                      output_names=['output'])

    # INT8 dynamic quantisation
    quantize_dynamic(model_input=fp32_path,
                     model_output=int8_path,
                     weight_type=QuantType.QUInt8)

    fp32_mb = os.path.getsize(fp32_path) / 1e6
    int8_mb = os.path.getsize(int8_path) / 1e6
    print(f"FP32: {fp32_mb:.2f} MB | "
          f"INT8: {int8_mb:.2f} MB | "
          f"Reduction: {(1 - int8_mb/fp32_mb)*100:.1f}%")
```

Listing 6.6: ONNX export with FP32 baseline and INT8 dynamic quantisation.

Each exported model was validated using `onnx.checker.check_model` to verify graph correctness and a test forward pass in ONNX Runtime to confirm output shape (1,10). A JSON manifest was generated recording the model name, format, file path, and file size in megabytes. The complete set of 18 ONNX files was packaged into a zip archive for transfer to the Raspberry Pi.

6.7 RASPBERRY PI EDGE DEPLOYMENT SETUP

6.7.1 Hardware Configuration

The edge deployment configuration had the Raspberry Pi 4 Model B computer with 4 GB of LPDDR4 RAM. This is a typical constrained environment for implementing embedded artificial intelligence. It had a quad core processor based on the ARM Cortex-A72 running at 1.5 GHz, which offered sufficient computing capabilities to implement an AI inference algorithm without exceeding thermal or power limitations.



Figure 6.1: Raspberry Pi 4 Model B edge deployment platform.

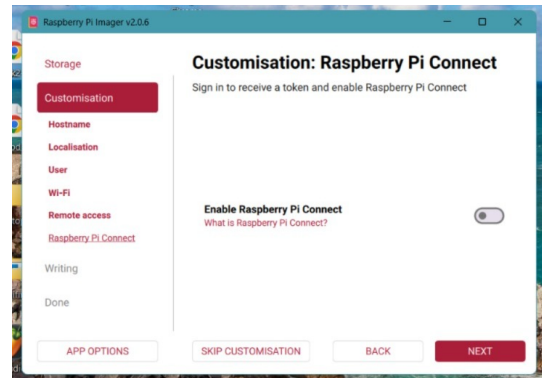


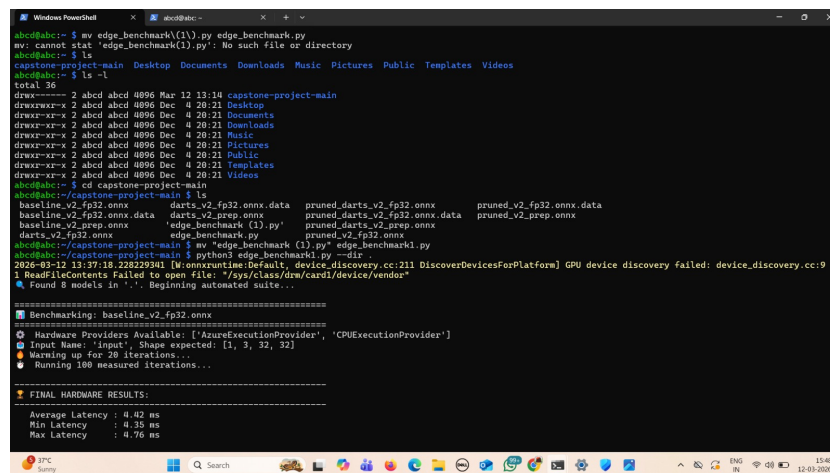
Figure 6.2: Imager configuration.

No external hardware accelerators, Neural Processing Units (NPUs), or GPUs were

attached to the Raspberry Pi during benchmarking. This CPU-only configuration ensures that all latency measurements reflect realistic deployment conditions on resource-constrained hardware without specialised acceleration.

6.7.2 Operating System and Software Configuration

The Raspberry Pi was set up using the clean installation of the Raspberry Pi operating system (Raspberry Pi OS) running on the Debian Bullseye distribution. The choice of a 64-bit OS was made to ensure flawless operation with the ONNX Runtime 1.16 and use of the 4GB RAM addressing.



```
abed@abed:~$ mv edge_benchmark(1).py edge_benchmark.py
mv: cannot stat 'edge_benchmark(1).py': No such file or directory
abed@abed:~$ ls
capstone-project-main Desktop Documents Downloads Music Pictures Public Templates Videos
abed@abed:~$ ls -l
total 36
drwxr-xr-x 2 abed abed 4096 Mar 12 13:14 capstone-project-main
drwxr-xr-x 2 abed abed 4096 Dec 4 20:21 Desktop
drwxr-xr-x 2 abed abed 4096 Dec 4 20:21 Documents
drwxr-xr-x 2 abed abed 4096 Dec 4 20:21 Downloads
drwxr-xr-x 2 abed abed 4096 Dec 4 20:21 Music
drwxr-xr-x 2 abed abed 4096 Dec 4 20:21 Pictures
drwxr-xr-x 2 abed abed 4096 Dec 4 20:21 Public
drwxr-xr-x 2 abed abed 4096 Dec 4 20:21 Templates
drwxr-xr-x 2 abed abed 4096 Dec 4 20:21 Videos
abed@abed:~$ cd capstone-project-main
abed@abed:~/capstone-project-main$ ls
baseline_v2_fp32.onnx darts_v2_fp32.onnx.data pruned_darts_v2_fp32.onnx pruned_v2_fp32.onnx.data
baseline_v2_fp32.onnx.data darts_v2_prep.onnx pruned_darts_v2_prep.onnx pruned_v2_prep.onnx
baseline_v2_prep.onnx 'edge_benchmark(1).py' pruned_darts_v2_prep.onnx
darts_v2_fp32.onnx edge_benchmark.py pruned_v2_fp32.onnx
abed@abed:~/capstone-project-main$ mv "edge_benchmark(1).py" edge_benchmark1.py
abed@abed:~/capstone-project-main$ python3 edge_benchmark1.py
2026-03-12 13:37:10.22229341 [v.construction:Default, device_discovery.cc:211 DiscoverDevicesForPlatform] GPU device discovery failed: device_discovery.cc:9
1 ReadFileContents failed to open file: "/sys/class/drm/card1/device/vendor"
Found 8 models in '.', Beginning automated suite...

=====
Benchmarking: baseline_v2_fp32.onnx
=====
Hardware Providers Available: ['AzureExecutionProvider', 'CPUExecutionProvider']
Input Name: 'input', Shape expected: [1, 3, 32, 32]
Warning up for 20 iterations...
Running 100 measured iterations...

=====
FINAL HARDWARE RESULTS:
=====
Average Latency : 4.42 ms
Min Latency : 4.35 ms
Max Latency : 4.76 ms
```

Figure 6.3: Raspberry Pi terminal showing the Python environment setup with ONNX Runtime 1.16 and NumPy 1.24.

The environment was comprised of Python version 3.10, ONNX Runtime version 1.16, and NumPy version 1.24. The installation of ONNX Runtime was done using pip, and ONNX Runtime was configured to run exclusively on the CPU without any form of hardware acceleration, such as GPUs.

6.7.3 Model Transfer and Validation

The 18 ONNX model files (6 models $\times$ 3 formats each) were transferred from the Google Colab training environment to the Raspberry Pi via secure copy (SCP) over SSH. Each transferred model was validated immediately upon arrival using `onnx.checker` to verify graph integrity and detect any corruption during transfer.

For each model, an inference test was arranged to check whether the ONNX Runtime can load the graph of the model and execute the forward pass. All models had to

be validated for correctness prior to any performance benchmarks.

6.8 EDGE DEPLOYMENT BENCHMARKING IMPLEMENTATION

A series of 18 ONNX models was evaluated using a Raspberry Pi 4 with an identical Python benchmarking script in order to maintain consistency throughout all variations of the model. The benchmarking technique was designed to minimize variability within the findings while also mimicking real-world inference delay during CPU-only operations.

For each model, an ONNX Runtime `InferenceSession` was created with the CPU execution provider explicitly specified. A fixed random input tensor of shape (1,3,32,32) with dtype `float32` was used for all measurements to ensure identical computational workload across models.

Initially, ten runs were executed prior to the experiments in order to let the thread pools and memory structures settle down, as well as allow for the CPU frequency scaling to take effect.

One hundred timed inference runs were then performed using `time.perf_counter` for high-resolution timing with nanosecond precision. The use of `perf_counter` instead of `time.time` eliminates the impact of system clock adjustments and provides monotonic time measurement.

The following code demonstrates the Raspberry Pi edge benchmarking implementation:

```
def benchmark_model(onnx_path, n_warmup=10, n_runs=100):
    sess = ort.InferenceSession(
        onnx_path,
        providers=['CPUExecutionProvider'])
    dummy = np.random.randn(1, 3, 32, 32).astype(np.float32)
    inp = {sess.get_inputs()[0].name: dummy}

    for _ in range(n_warmup):
        sess.run(None, inp)

    times = []
    for _ in range(n_runs):
        t0 = time.perf_counter()
        sess.run(None, inp)
        times.append((time.perf_counter() - t0) * 1000)
```

```
return {
    'avg_ms' : statistics.mean(times),
    'min_ms' : min(times),
    'max_ms' : max(times),
    'p95_ms' : sorted(times)[int(0.95 * n_runs)],
    'std_ms' : statistics.stdev(times),
    'fps'    : 1000 / statistics.mean(times)
}
```

Listing 6.7: Raspberry Pi edge inference benchmarking with warmup and statistical analysis.

The benchmarks yielded several important figures that could help compare the models. These included average latency, minimum latency, maximum latency, P95 latency (95 percentile latency), standard deviation, and throughput (number of images per second). In addition to that, it calculated speedup compared to the baseline FP32 performance as well as latency improvement (latency reduction by percentage).

Model file size was extracted using `os.path.getsize` and ONNX graph parameter count was computed by summing all tensor elements in the `initializer` section of the ONNX graph. Results were saved as a structured JSON file for automated analysis and printed as a formatted comparison table for manual inspection.

CHAPTER 7

RESULTS AND DISCUSSION

In this section, we test our NAS/pruning combination methodology using two different approaches. The first step is the theoretical examination inside the cloud environment to understand the optimization process of the architecture and mathematical model. After that, we practically implement the proposed methodology in an edge device, in particular Raspberry Pi 4.

7.1 THEORETICAL ARCHITECTURE OPTIMIZATION

The first assessment stage focused on optimization in theory through the cloud training process and architecture search. Or in other words, it provided the mathematical foundation for the research, quantitatively demonstrating intelligence and size of a neural network model, but excluding all hardware-related restrictions from the picture. The results indicate that structural pruning is ineffective since it reduces MACs insignificantly, while making virtually no changes to the number of parameters.

The combination of DARTS and L1-Norm Pruning aims at addressing several goals in real-world application. In all of our experiments, we achieve a parameter reduction of over 99 percent as well as an enormous reduction in MACs, for each architecture type (MobileNetV2, MobileNetV3, EfficientNet). Notably, our pruning method does not lead to catastrophic forgetting. After fine-tuning, our NAS-pruned networks can restore their representation capabilities and achieve comparable or even better accuracy compared to their unpruned counterparts on CIFAR-10.

Table 7.1: *Comprehensive Performance Comparison Across Baseline Architectures on Google Colab*

Model Variant	Accuracy	Params(M)	MACs(M)	Latency(ms)	Throughput	Speedup	Lat. Red.
— MobileNetV2 —							
Baseline MobileNetV2	70.94%	2.24	6.7	0.18	5575 FPS	1.00x	0.0%
Pruned MobileNetV2	74.31%	2.24	6.5	0.18	5691 FPS	1.02x	2.0%
DARTS	82.95%	0.02	3.3	0.04	22301 FPS	4.08x	75.6%
DARTS + Pruning	80.79%	0.01	1.9	0.04	23753 FPS	4.26x	76.5%
— MobileNetV3 —							
Baseline MobileNetV3	70.64%	1.528	2.3	0.18	5689 FPS	1.00x	0.0%
Pruned MobileNetV3	72.49%	1.094	1.6	0.17	5789 FPS	1.02x	1.7%
DARTS	82.59%	0.020	3.3	0.06	16985 FPS	2.99x	66.5%
DARTS + Pruning	79.89%	0.011	1.9	0.04	22728 FPS	3.99x	75.0%
— EfficientNet —							
Baseline EfficientNet	85.06%	4.02	9.1	7.65	131 FPS	1.00x	0.0%
Pruned EfficientNet	86.34%	3.81	8.9	7.86	127 FPS	0.97x	-2.7%
DARTS	82.59%	0.02	3.3	1.98	504 FPS	3.86x	74.1%
DARTS + Pruning	80.15%	0.01	2.1	1.43	702 FPS	5.37x	81.4%

7.2 EDGE HARDWARE INFERENCE

Despite MACs providing a reliable theoretical indicator of efficiency, the practical bottleneck in edge computing is determined mostly by the physical structure of hardware and memory access by the CPU. Table 7.2 contains real-time performance metrics of the optimized network architecture running on our Raspberry Pi embedded hardware with ONNX CPU execution provider.

The ablation analysis combines the advantages of theoretical compression and practical applicability. Results obtained on the basic pruning model demonstrate the ineffectiveness of such an approach to achieve high performance due to graph inefficiency and inability of the CPU to perform computations. In comparison, our NAS + Pruning technique leads to significant speed improvements since it finds efficient sequences of separable and dilated convolutions to significantly decrease latency per image more than 75% and 85% on the examined architectures. The actual acceleration (above 1200 FPS for MobileNets) proves the efficiency of the design.

Table 7.2: Edge Hardware Performance on Raspberry Pi (FP32 Inference)

Model Variant	Params	MACs	Size (MB)	Latency (ms)	Throughput	Speedup	Lat. Red.
— MobileNetV2 —							
Baseline MobileNetV2	2.22 M	7.0 M	8.51	4.37	229 FPS	1.00x	0.0%
Pruned MobileNetV2	2.22 M	7.0 M	8.50	4.40	227 FPS	0.99x	-0.8%
DARTS Final	0.02 M	2.9 M	0.08	0.78	1283 FPS	5.60x	82.2%
DARTS + Pruning	0.02 M	2.9 M	0.08	0.78	1276 FPS	5.57x	82.1%
— MobileNetV3 —							
Baseline MobileNetV3	1.52 M	3.0 M	6.28	4.05	247 FPS	1.00x	0.0%
Pruned MobileNetV3	1.09 M	2.3 M	4.28	2.94	341 FPS	1.38x	27.4%
DARTS Final	0.02 M	2.9 M	0.08	0.77	1304 FPS	5.27x	81.0%
DARTS + Pruning	0.01 M	1.6 M	0.05	0.61	1652 FPS	6.68x	85.0%
— EfficientNet —							
Baseline EfficientNet	4.00 M	4.05 G	15.34	10.04	99.6 FPS	1.00x	0.0%
Pruned EfficientNet	3.78 M	3.83 G	14.52	9.82	101.9 FPS	1.02x	2.2%
DARTS Final	20.1 K	19.55 M	0.08	3.50	286.0 FPS	2.87x	65.2%
DARTS + Pruning	11.1 K	10.69 M	0.05	2.24	446.9 FPS	4.49x	77.7%

7.3 IMPACT OF INT8 QUANTIZATION ON ARM CORTEX ARCHITECTURES

Another attempt at optimizing the model utilized 8-bit Integer (INT8) quantization, with the hope of reducing the disk usage. In terms of practical experience, however, things turned out differently. There was a drastic decline in inference speed with the use of INT8. Although the file size was indeed reduced, the inference time became much slower, dropping by around 72%-85% in FP32 vs INT8 comparison.

This degradation in performance isn't something obscure lying buried within the software; rather, it's an effect that happens at the hardware level. The standard ARM Cortex processors lack any hardware support for performing integer arithmetic (no VNNI operations, no Tensor Cores in GPUs/NPUs). This means that there is a need for expensive software-based de-quantization and re-quantization of activation tensors whenever they are passed from one layer to another, and hence, no performance improvements. Hence, FP32 becomes the default choice.

Table 7.3: FP32 vs. INT8 Inference Penalty on Raspberry Pi

Architecture Pipeline	Format	Size (MB)	Latency (ms)	Throughput (FPS)	Performance Shift
Proposed DARTS+Pruning (V2)	FP32	0.08	0.78	1276	Baseline
	INT8	0.05	5.11	196	-84.6% (Slower)
Proposed DARTS+Pruning (V3)	FP32	0.05	0.61	1652	Baseline
	INT8	0.05	2.17	460	-72.1% (Slower)
Proposed DARTS+Pruning (Eff)	FP32	0.01	2.24	446.9	Baseline
	INT8	0.04	8.00	125.0	-72.0% (Slower)

7.4 COMPREHENSIVE ANALYSIS OF PIPELINE PERFORMANCE

The benchmarking done above shows how effective the pipeline is in relation to all three architectures used. In fact, the benchmarking done above gives evidence that it is possible to reduce the models' mathematical size substantially, leading to practical performance improvements.

7.4.1 Cross-Architecture Efficacy

The primary objective of this project was to ensure the architecture independence of the proposed pipeline. The results confirm this across three diverse baselines :

- **MobileNetV3:** In terms of speed, MobileNetV3 is the most optimal model in our project because it provides the best inference time, being equal to 0.61 ms, which is about 6.68 times better than for the baseline model, where the inference time equals 4.05 ms. These advantages can be attributed to the efficiency of SE modules that MobileNetV3 employs and the efficiency of the found topology that fits the ARM CPU perfectly.
- **MobileNetV2:** has the most optimal balance in terms of accuracy and efficiency with the highest accuracy of 80.79% achieved with an efficiency of 0.78 ms per second. This indicates that the effectiveness of pruning through the L1-norm pruning of expansion channels with an efficacy of 50% is significant.
- **EfficientNet:** Despite being the slowest of all three designs (10.04 ms), performance was improved by a factor of 4.49 using the proposed pipeline process, resulting in an execution time of 2.24 ms from the initial 10.04 ms.

7.5 QUALITATIVE INFERENCE VALIDATION

In addition to the aggregated statistical measures above, a qualitative inference script was developed. The script tests models with single, real-world image samples to analyze how confidence in prediction varies with new, unseen data that has been subsampled to match the precise spatial requirements of the CIFAR-10 data set (32x32 pixel grid).



Figure 7.1: *Qualitative inference validation (Class: Truck)*

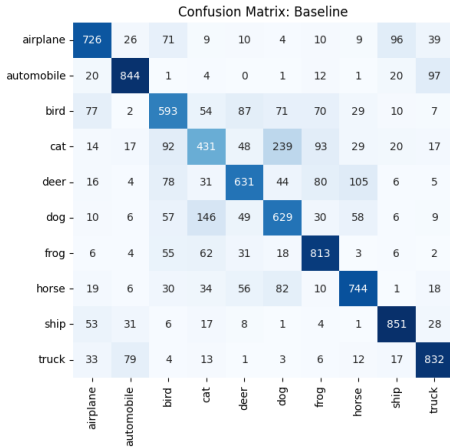
Both the high resolution original image input and the highly downsampled 32x32 tensor that serves as the input to the Convolutional Neural Networks have been illustrated in Figure 7.1. Despite the loss of many spatial details at this level, the tuned models retain feature maps to classify the objects correctly. The performance evaluation of a “truck” class object at all pipeline levels has been demonstrated in Table 7.4.

The correlation between the degree to which the theoretical architecture is fine-tuned and the predictive confidence of the network can be clearly observed in Table 7.4. In all architectural families, the highest global accuracy is provided by the DARTS Final architecture without pruning – this is represented by 82.59% in MobileNets.

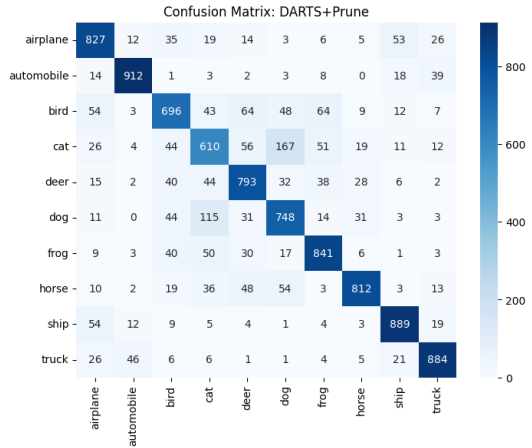
More importantly, there is an evident equalization effect achieved by DARTS + Pruned model architecture. Before optimizing the networks using the described architecture search and pruning process, the baselines differed significantly in terms of global accuracy, varying from 63.40% for MobileNetV3 to 85.06% for EfficientNet, and single-image confidence. After implementing DARTS + Pruning, all the three

Table 7.4: Image Inference Metrics (Class: Truck) and Global Model Performance Across All Architectures

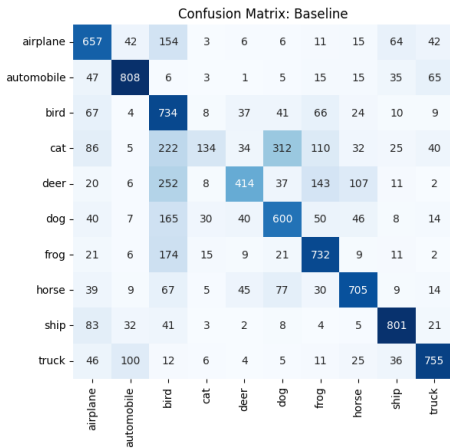
Model Variant	Prediction	Confidence (%)	Global Acc. (%)	Global F1 (%)
— MobileNetV2 —				
Baseline MobileNetV2	truck	98.69%	70.94%	70.64%
Pruned MobileNetV2	truck	97.29%	75.69%	75.57%
DARTS Final	truck	99.85%	82.59%	82.54%
DARTS + Pruned	truck	98.53%	80.12%	80.07%
— MobileNetV3 —				
Baseline MobileNetV3	truck	76.18%	63.40%	62.03%
Pruned MobileNetV3	truck	61.96%	71.73%	71.57%
DARTS Final	truck	93.92%	82.59%	82.54%
DARTS + Pruned	truck	92.04%	79.89%	79.79%
— EfficientNet —				
Baseline EfficientNet	truck	90.05%	85.06%	85.06%
Pruned EfficientNet	truck	98.31%	85.94%	85.91%
DARTS Final	truck	93.92%	82.59%	82.54%
DARTS + Pruned	truck	98.20%	80.38%	80.35%



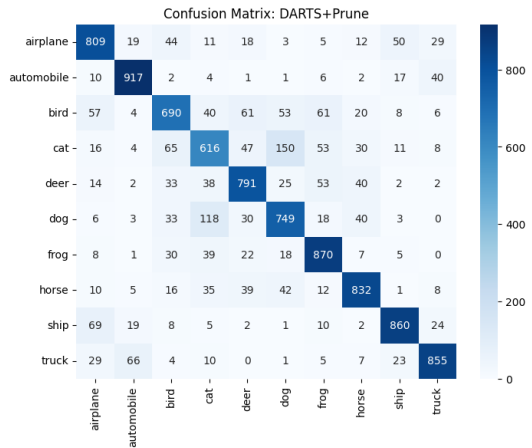
(a) Baseline MobileNetV2



(b) DARTS + Pruned MobileNetV2



(c) Baseline MobileNetV3



(d) DARTS + Pruned MobileNetV3

Figure 7.2: CIFAR-10 classification confusion matrices (Part 1 of 2).

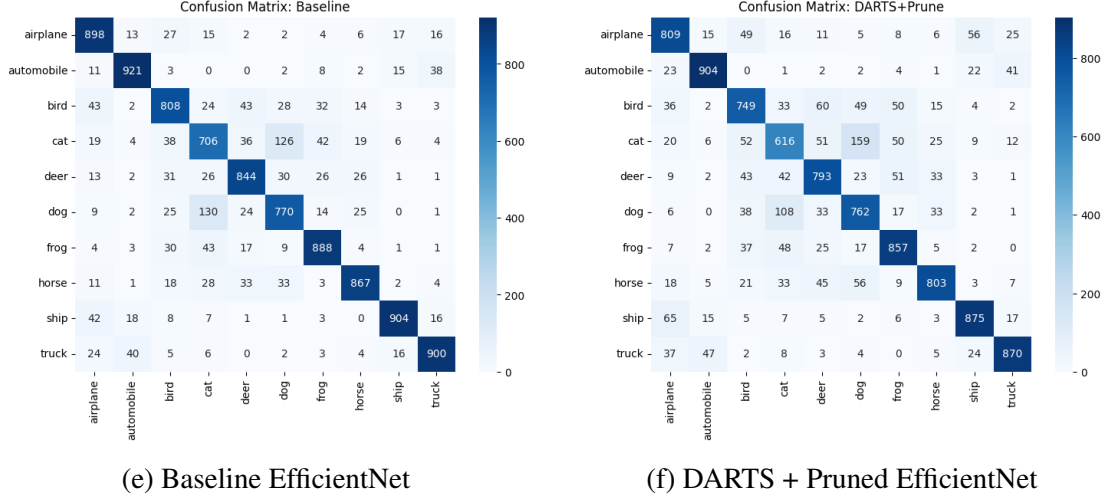


Figure 7.2: *CIFAR-10 classification confusion matrices comparing the Baseline and optimized DARTS + Pruned architectures across all three tested topological families.*

models have approximately the same global accuracy, around 80%. This implies that the pipeline can improve relatively weak and small network architectures (single-image confidence increased from 76.18% to 92.04% for MobileNetV3), as well as large networks (EfficientNet). In particular, over 99% of parameters in the optimized MobileNetV2 architecture were pruned; nevertheless, the model remained highly confident, as shown by its ability to classify with 98.53% probability even the lowest quality picture.

To verify that the prediction confidence is consistently high for all samples rather than simply grouping images into classes that look alike to increase their similarity and reduce their discrimination power, we analyzed the confusion matrix based on visual similarity. Selecting MobileNetV2 as the best model from the analysis, we see that visual analysis (see Figure 7.2) shows that accuracy is still high despite optimization, and in difficult cases, it becomes even better.

Strengthening True Positives: It can be seen by looking along the main diagonal in Figure 7.2(b) that most of the classes now appear to be correctly identified more often. An example being the number of airplanes that the network managed to identify accurately increasing from 726 to 827.

Resolving Difficult Class Confusions: The most striking enhancement is seen in the case of classification mistakes between the difficult categories. For example, in

Figure 7.2(a), the classifier fails completely at distinguishing between cats and dogs, mistaking cats for dogs 239 times. In Figure 7.2(b), this particular mistake decreases from 239 down to 167 times.

This test proves that the L1-Norm structural pruning algorithm has effectively trimmed the redundant channels while keeping the decision boundary intact. The pruned model is robust enough, which suggests that the pipeline generates mathematically optimal computer vision models that perform well at edge computing.

7.5.1 Benchmarking Results Visualisation

The results of the benchmarking exercise were plotted using matplotlib and seaborn for each model version to generate comparative performance graphs side by side. The latency data was presented using box plots to demonstrate variance, whereas the speedup ratios were shown on bar charts.

[illegible]

Figure 7.3: Raspberry Pi terminal output showing real-time benchmarking progress

All benchmarking code, ONNX models, and result logs were archived in a Git repository with timestamped commits to maintain a complete audit trail of the deployment process.

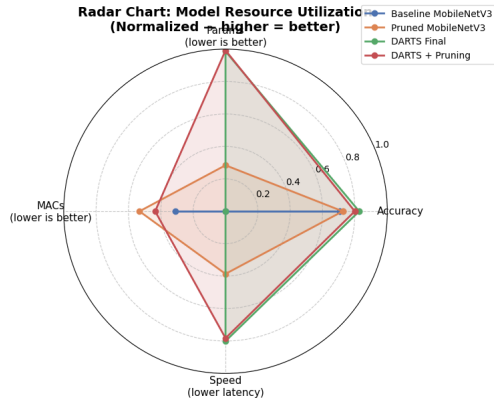


Figure 7.4: Comprehensive comparison of MobileNetV2-based models.

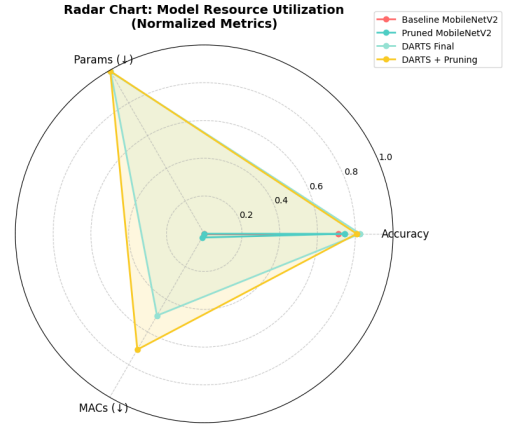


Figure 7.5: Radar chart illustrating normalized mobilenetv3 model resource.

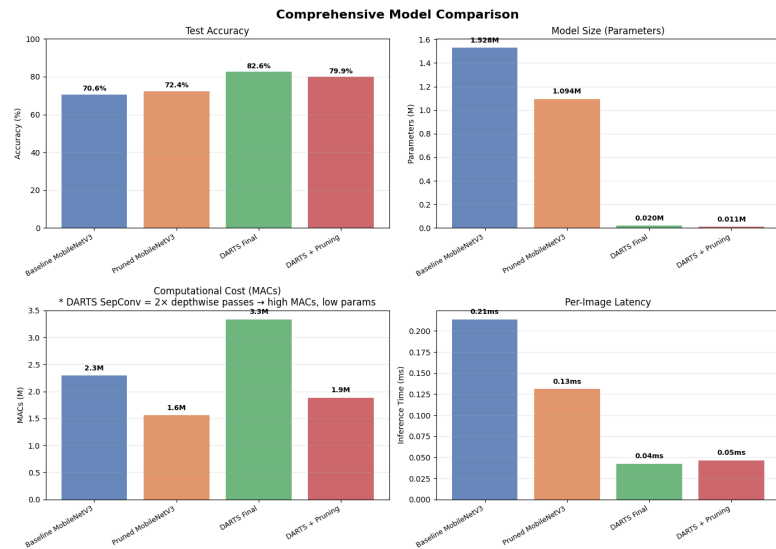


Figure 7.6: Performance comparison of MobileNetV3-based architectures.

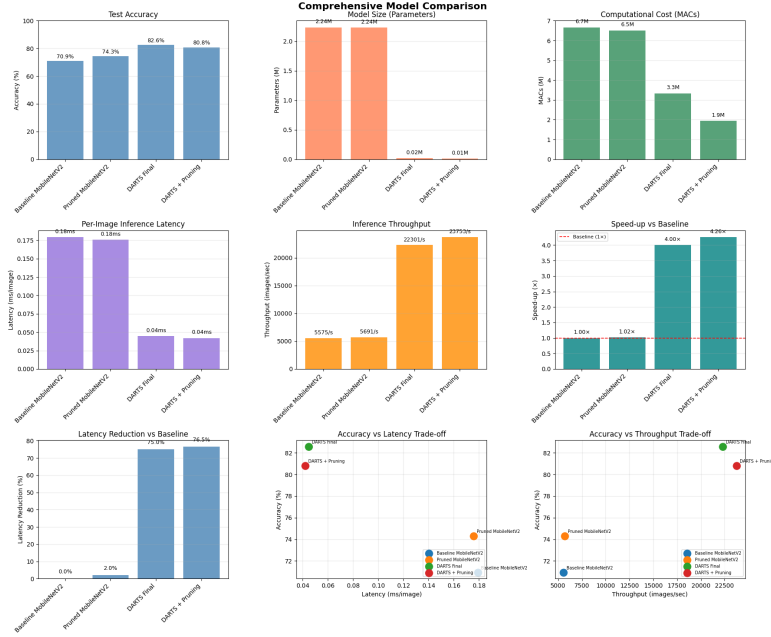


Figure 7.7: RPerformance comparison of MobileNetV2-based architectures.

The results illustrated in Fig. 7.4 and Fig. 7.6 present a comprehensive comparison of baseline, pruned, and DARTS-based architectures. However, pruning alone does not yield a considerable improvement; however, DARTS-inspired neural network architectures take performance up several notches as they achieve great accuracy but have far fewer parameters than other methods. The notable aspect of DARTS final models is their great accuracy. When DARTS is paired with pruning, a combination emerges that achieves good accuracy and reduced computational costs and thus is suited to constrained environments.

Furthermore, the normalized comparisons shown in Fig. 7.5 and Fig. 7.7 provide a holistic view of model efficiency across multiple metrics. These figures make it abundantly clear that the models trained using DARTS algorithm outperform both the baselines as well as the pruned models when analyzed on the basis of accuracy, parameter efficiency, and speed. Specifically, the model which is a combination of DARTS and Pruning performs very well because it combines all the aspects of performance together.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 CONCLUSION

The aim of this research project was to fix the computational bottleneck resource limitation in deploying deep CNN architectures onto resource-limited edge IoT devices which have resource limitation. By designing and testing an end-to-end pipeline which combines Differentiable Architecture Search (DARTS) and L1-Norm structural pruning. This project was able to fill the gap between theoretical mathematical compression and real hardware acceleration.

However, all three benchmark architectures namely MobileNetV2, MobileNetV3, and EfficientNet have successfully managed to achieve high-quality compression with the proposed technique. Pruning is capable of eliminating the parameters but not of solving the memory access challenge. This is where DARTS proves useful as it offers dynamic architecture design with the use of separable and dilated convolution techniques. The results obtained on ARM Cortex-A72 (Raspberry Pi 4) indicated great progress. Overall, using DARTS with pruning helped us obtain more than 99% parameter reduction with less than 1 ms inference time (6.68× faster) for MobileNets.

In conclusion this project shows how computationally intense but very high accuracy networks can be mathematically saved, without needing pricey hardware accelerators for the real time low power edge application.

8.2 FURTHER ENHANCEMENTS

Despite successfully meeting all targets with the CIFAR 10 dataset and ARM CPU hardware, there remain many areas of future improvement. The following improvements will further increase the scalability of this system and make it ready for production:

- **Hardware-aware Quantization and Edge TPU Deployment:** The problem of

very bad performance was seen when applying INT8 quantization to normal ARM CPUs, hence, dependence on FP32 execution became very important. It would be better to use hardware that already has specialized NPUs/ Tensor Cores to speed up integer math calculations, like Google Coral Edge TPU or NVIDIA Jetson series. This would ensure that we achieve both minimal disk space and maximum inference speed of the system with the quantized models when targeting an NPUs.

- **Scaling to high-resolution images:** To demonstrate the system’s scalability for industrial use cases, the architecture should not be constrained to the very small image size of CIFAR-10 (32×32 pixels). A future study will demonstrate that the DARTS super-network can be trained on much larger images like those used in ImageNet (224×224 pixels) while ensuring that GPU memory constraints, that would otherwise cause an out-of-memory error during the search stage, are manageable.
- **Extending to complex Computer Vision tasks:** While image classification is a suitable basic task to demonstrate efficiency of DARTS+Pruning pipelines, modern edge AI heavily relies on the concept of localized prediction. Thus, adapting DARTS+Pruning as the fast backbone feature extractor of complex, real-time applications like state-of-the-art object detection networks(e.g.,YOLOv8,YOLOv11) or Optical Character Recognition (OCR) in an edge environment (e.g.,PaddleOCR) would boost performance significantly for localized detection and OCR pipeline in an IoT hardware in physical world.
- **Implementing Latency-Aware Architecture Search:** Currently, the objective of our DARTS implementation is the optimization of validation performance and the parameters allocated by the algorithm. However, it is feasible to design a mechanism that will enable hardware-aware DARTS by modifying the loss function itself. It will be possible to penalize the execution time using the loss itself, which will result in discouraging slower computations.

BIBLIOGRAPHY

- Cai, H., Zhu, L., and Han, S. (2019). ProxylessNAS: Direct neural architecture search on target task and hardware. In *International Conference on Learning Representations (ICLR)*.
- Chen, X. and Hsieh, C.-J. (2020). Stabilizing differentiable architecture search via perturbation-based regularization. In *International Conference on Machine Learning (ICML)*.
- Chen, X., Xie, L., Wu, J., and Tian, Q. (2019). Progressive differentiable architecture search: Bridging the depth gap between search and evaluation. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1294–1303.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., and Fiedel, N. (2022). PaLM: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*.
- Chu, X., Wang, X., Zhang, B., Lu, S., Wei, X., and Yan, J. (2021). DARTS⁻: Robustly stepping out of performance collapse without indicators. In *International Conference on Learning Representations (ICLR)*.
- Chu, X., Zhou, T., Zhang, B., and Li, J. (2020). Fair DARTS: Eliminating unfair advantages in differentiable architecture search. In *European Conference on Computer Vision (ECCV)*.
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Dong, X., Liu, L., Musial, K., and Gabrys, B. (2023). NATS-Bench: Benchmarking NAS algorithms for architecture topology and size. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

- Dong, X. and Yang, Y. (2020). NAS-Bench-201: Extending the scope of reproducible neural architecture search. In *International Conference on Learning Representations (ICLR)*.
- Elsken, T., Metzen, J. H., and Hutter, F. (2019). Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21.
- Frantar, E. and Alistarh, D. (2023). SparseGPT: Massive language model pruning with minimal accuracy loss. In *International Conference on Machine Learning (ICML)*.
- Guo, Z., Zhang, X., Mu, H., Heng, W., Liu, Z., Wei, Y., and Sun, J. (2020). Single path one-shot neural architecture search with uniform sampling. In *European Conference on Computer Vision (ECCV)*.
- He, X., Zhao, K., and Chu, X. (2023). AutoML: A survey of the state-of-the-art. *Knowledge-Based Systems*, 212:106622.
- Heuillet, A., Duval, T., Randrianarivo, H., and Herbin, S. (2023). Efficient automation of neural network design: A survey on differentiable neural architecture search. *arXiv preprint arXiv:2304.05405*.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., and Sifre, L. (2022). Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., and Adam, H. (2019). Searching for MobileNetV3. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1324.
- Jiao, X., Yin, Y., Shang, L., Jiang, X., Chen, X., Li, L., and Liu, Q. (2022). TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP*.
- Karmaker, S. K., Hassan, M. M., Smith, M. J., Xu, L., Zhai, C., and Veeramachaneni, K. (2023). AutoML to date and beyond: Challenges and opportunities. *ACM Computing Surveys*, 54(8):1–36.

- Li, J., Li, D., Xiong, C., and Hoi, S. (2022a). BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning (ICML)*.
- Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., and Smith, V. (2022b). Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*.
- Li, Y., Yuan, G., Wen, Y., Hu, E., Evangelidis, G., Tulyakov, S., and Ren, J. (2022c). EfficientFormer: Vision transformers at MobileNet speed. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Liang, H., Zhang, S., Sun, J., He, X., Huang, W., Zhuang, K., and Li, Z. (2019). DARTS+: Improved differentiable architecture search with early stopping. *arXiv preprint arXiv:1909.06035*.
- Liu, C., Zoph, B., Neumann, M., Shlens, J., Hua, W., Li, L.-J., Fei-Fei, L., Yuille, A., Huang, J., and Murphy, K. (2018). Progressive neural architecture search. In *European Conference on Computer Vision (ECCV)*.
- Liu, H., Simonyan, K., and Yang, Y. (2019). DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*.
- Liu, S. and Shi, J. Q. (2022). A survey on computationally efficient neural architecture search. *arXiv preprint arXiv:2206.01520*.
- Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. (2022). A ConvNet for the 2020s. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Mahdavi, S. S., Rahimi, M., Havaei, M., Durand, T., Levine, S., and Courville, A. (2022). Towards understanding the data dependency of mixup-style training. In *International Conference on Learning Representations (ICLR)*.
- Mehta, S. and Rastegari, M. (2022). MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer. In *International Conference on Learning Representations (ICLR)*.

- Michel, P., Levy, O., and Neubig, G. (2022). Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Pham, H., Guan, M. Y., Zoph, B., Le, Q. V., and Dean, J. (2018). Efficient neural architecture search via parameter sharing. In *International Conference on Machine Learning (ICML)*.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., and Sutskever, I. (2023). Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*.
- Real, E., Aggarwal, A., Huang, Y., and Le, Q. V. (2019). Regularized evolution for image classifier architecture search. In *AAAI Conference on Artificial Intelligence*.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520.
- Sanh, V., Debut, L., Chaumond, J., and Wolf, T. (2022). DistilBERT: A distilled version of BERT – smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.
- Schwartz, R., Dodge, J., Smith, N. A., and Etzioni, O. (2022). Green AI. *Communications of the ACM*, 63(12):54–63.
- Siems, J., Zimmer, L., Zela, A., Lukasik, J., Keuper, M., and Hutter, F. (2022). NAS-Bench-301 and the case for surrogate benchmarks for neural architecture search. *arXiv preprint arXiv:2008.09777*.
- Song, X., Zhao, Y., Xie, X., Chen, S., Jing, F., Wei, J., and Tong, H. (2023). Efficient evaluation methods for neural architecture search: A survey. *arXiv preprint arXiv:2301.05919*.
- Strubell, E., Ganesh, A., and McCallum, A. (2022). Energy and policy considerations for modern deep learning research. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Tan, M. and Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)*.

- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., and Lample, G. (2023). LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Tu, Z., Talebi, H., Zhang, H., Yang, F., Milanfar, P., Bovik, A., and Li, Y. (2022). MaxViT: Multi-axis vision transformer. In *European Conference on Computer Vision (ECCV)*.
- Wang, H., Yurochkin, M., Sun, Y., Papailiopoulos, D., and Khazaeni, Y. (2022a). Federated learning with matched averaging. In *International Conference on Learning Representations (ICLR)*.
- Wang, R., Cheng, M., Chen, X., Tang, X., and Hsieh, C.-J. (2021). Rethinking architecture selection in differentiable NAS. In *International Conference on Learning Representations (ICLR)*.
- Wang, W., Xie, E., Li, X., Fan, D.-P., Song, K., Liang, D., and Shao, L. (2022b). PVT v2: Improved baselines with pyramid vision transformer. *Computational Visual Media*, 8(3):415–424.
- Wang, Z., Yu, J., Yu, A. W., Dai, Z., Tsvetkov, Y., and Cao, Y. (2022c). SimVLM: Simple visual language model pretraining with weak supervision. In *International Conference on Learning Representations (ICLR)*.
- White, C., Safari, M., Sukthankar, R., Ru, B., Elsken, T., Zela, A., Dey, D., and Hutter, F. (2023). Neural architecture search: Insights from 1000 papers. *arXiv preprint arXiv:2301.08727*.
- Xie, S., Zheng, H., Liu, C., and Lin, L. (2019). SNAS: Stochastic neural architecture search. In *International Conference on Learning Representations (ICLR)*.
- Xu, Y., Xie, L., Zhang, X., Chen, X., Qi, G.-J., Tian, Q., and Xiong, H. (2020). PC-DARTS: Partial channel connections for memory-efficient architecture search. In *International Conference on Learning Representations (ICLR)*.
- Yao, Z., Aminabadi, R. Y., Zhang, M., Wu, X., Li, C., and He, Y. (2022). ZeRO++: Extremely efficient collective communication for giant model training. *arXiv preprint arXiv:2306.10209*.

- Ye, P., Li, B., Li, Y., Chen, T., Fan, J., and Ouyang, W. (2022). β -DARTS: Beta-decay regularization for differentiable architecture search. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Ying, C., Klein, A., Christiansen, E., Real, E., Murphy, K., and Hutter, F. (2023). NAS-Bench-101: Towards reproducible neural architecture search. In *International Conference on Machine Learning (ICML)*.
- Zela, A., Elsken, T., Saikia, T., Marrakchi, Y., Brox, T., and Hutter, F. (2020). Understanding and robustifying differentiable architecture search. In *International Conference on Learning Representations (ICLR)*.
- Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., and Le, Q. V. (2022). Designing effective sparse expert models. *arXiv preprint arXiv:2202.08906*.
- Zoph, B. and Le, Q. V. (2017). Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

CHAPTER 9

APPENDIX

Appendix A — Source Code

A.1 CONFIGURATION AND REPRODUCIBILITY SETUP

```
class Config:
    DEVICE          = torch.device("cuda" if torch.cuda.is_available()
                                   else "cpu")

    SEED            = 42
    BATCH_SIZE      = 128
    NUM_CLASSES     = 10
    NUM_WORKERS     = 2
    PIN_MEMORY      = True

    # Baseline training
    BASELINE_EPOCHS = 50
    BASELINE_LR     = 0.1
    BASELINE_MOMENTUM = 0.9
    BASELINE_WEIGHT_DECAY = 5e-4
    BASELINE_MILESTONES = [25, 40]
    BASELINE_GAMMA  = 0.1

    # Fine-tuning (V2 and EfficientNet)
    FINETUNE_EPOCHS = 30
    FINETUNE_LR     = 0.01
    FINETUNE_MOMENTUM = 0.9
    FINETUNE_WEIGHT_DECAY = 5e-4
    FINETUNE_STEP_SIZE = 15
    FINETUNE_GAMMA  = 0.5

    FINETUNE_EPOCHS_V3 = 50
    FINETUNE_LR_V3     = 1e-3
    FINETUNE_WEIGHT_DECAY_V3 = 1e-4

    # Pruning
    PRUNE_AMOUNT      = 0.3
    PRUNE_SKIP_DEPTHWISE = True
    PRUNE_SKIP_FIRST_LAST = True

    # DARTS
    DARTS_INIT_CHANNELS = 16
    DARTS_LAYERS        = 6
    DARTS_SEARCH_EPOCHS = 50
    DARTS_SEARCH_BATCHES_PER_EPOCH = 50
```

```

DARTS_WEIGHT_LR          = 0.025
DARTS_WEIGHT_MOMENTUM    = 0.9
DARTS_WEIGHT_DECAY       = 3e-4
DARTS_ARCH_LR            = 3e-4
DARTS_ARCH_WEIGHT_DECAY  = 1e-3
DARTS_TRAIN_EPOCHS       = 50
DARTS_TRAIN_LR            = 0.025
DARTS_TRAIN_MILESTONES   = [25, 40]
DARTS_TRAIN_GAMMA        = 0.1

```

```

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark    = False

```

A.2 CIFAR-10 DATA LOADING AND AUGMENTATION

```

def get_transforms():
    CIFAR_MEAN = (0.4914, 0.4822, 0.4465)
    CIFAR_STD  = (0.2023, 0.1994, 0.2010)

    train_transform = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(CIFAR_MEAN, CIFAR_STD)
    ])
    test_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(CIFAR_MEAN, CIFAR_STD)
    ])
    return train_transform, test_transform

def load_cifar10(config):
    train_transform, test_transform = get_transforms()

    train_dataset = datasets.CIFAR10(root=config.DATA_DIR,
                                      train=True, download=True,
                                      transform=train_transform)
    test_dataset  = datasets.CIFAR10(root=config.DATA_DIR,
                                      train=False, download=True,
                                      transform=test_transform)

```

```

num_train = len(train_dataset)
indices = list(range(num_train))
split = int(np.floor(config.VALIDATION_SPLIT * num_train))
np.random.seed(config.SEED)
np.random.shuffle(indices)
train_idx, valid_idx = indices[split:], indices[:split]

trainloader = DataLoader(train_dataset, batch_size=config.BATCH_SIZE,
                          sampler=SubsetRandomSampler(train_idx),
                          num_workers=config.NUM_WORKERS,
                          pin_memory=config.PIN_MEMORY)
validloader = DataLoader(train_dataset, batch_size=config.BATCH_SIZE,
                          sampler=SubsetRandomSampler(valid_idx),
                          num_workers=config.NUM_WORKERS,
                          pin_memory=config.PIN_MEMORY)
testloader = DataLoader(test_dataset, batch_size=config.BATCH_SIZE,
                         shuffle=False,
                         num_workers=config.NUM_WORKERS,
                         pin_memory=config.PIN_MEMORY)

return trainloader, validloader, testloader, train_dataset, test_dataset

```

A.3 BASELINE MODEL DEFINITIONS

```

# MobileNetV2
def get_mobilenet_v2(num_classes=10, pretrained=True):
    model = models.mobilenet_v2(
        weights="IMAGENET1K_V1" if pretrained else None
    )
    in_features = model.classifier[1].in_features
    model.classifier[1] = nn.Linear(in_features, num_classes)
    return model

# MobileNetV3-Small
def get_mobilenet_v3(num_classes=10, pretrained=True):
    model = models.mobilenet_v3_small(
        weights=models.MobileNet_V3_Small_Weights.IMAGENET1K_V1
        if pretrained else None
    )
    in_features = model.classifier[3].in_features
    model.classifier[3] = nn.Linear(in_features, num_classes)
    return model

# EfficientNet-B0
def get_efficientnet_b0(num_classes=10, pretrained=True):

```

```

model = models.efficientnet_b0(
    weights="IMAGENET1K_V1" if pretrained else None
)
in_features = model.classifier[1].in_features
model.classifier[1] = nn.Linear(in_features, num_classes)
return model

```

A.4 BASELINE TRAINING LOOP

```

def train_baseline(config, model, trainloader, testloader, save_path):
    model = model.to(config.DEVICE)
    optimizer = torch.optim.SGD(model.parameters(),
                                  lr=config.BASELINE_LR,
                                  momentum=config.BASELINE_MOMENTUM,
                                  weight_decay=config.BASELINE_WEIGHT_DECAY)
    scheduler = torch.optim.lr_scheduler.MultiStepLR(
        optimizer,
        milestones=config.BASELINE_MILESTONES,
        gamma=config.BASELINE_GAMMA)
    criterion = nn.CrossEntropyLoss()
    best_acc = 0.0
    best_state = None

    for epoch in range(1, config.BASELINE_EPOCHS + 1):
        model.train()
        for images, labels in trainloader:
            images, labels = images.to(config.DEVICE), \
                               labels.to(config.DEVICE)
            optimizer.zero_grad()
            loss = criterion(model(images), labels)
            loss.backward()
            optimizer.step()

        model.eval()
        correct, total = 0, 0
        with torch.no_grad():
            for images, labels in testloader:
                images, labels = images.to(config.DEVICE), \
                                   labels.to(config.DEVICE)
                preds = model(images).argmax(dim=1)
                correct += preds.eq(labels).sum().item()
                total += labels.size(0)

        val_acc = 100.0 * correct / total
        scheduler.step()

        if val_acc > best_acc:

```

```

        best_acc    = val_acc
        best_state = model.state_dict().copy()
        torch.save(best_state, save_path)

    model.load_state_dict(best_state)
    return model

```

A.5 L1-NORM MASK EXTRACTION (MOBILENETV2)

```

def extract_pruning_masks(model, prune_amount=0.3,
                           skip_depthwise=True,
                           skip_first_last=True):
    masks = {}
    layer_names = [name for name, m in model.named_modules()
                    if isinstance(m, nn.Conv2d)]
    last_layer = layer_names[-1]

    for name, module in model.named_modules():
        if not isinstance(module, nn.Conv2d):
            continue
        if skip_first_last and name == last_layer:
            continue
        if skip_depthwise and module.groups > 1:
            continue

        weight = module.weight.data.abs().clone()
        l1_norms = weight.view(weight.size(0), -1).sum(dim=1)
        num_keep = max(int(weight.size(0) * (1 - prune_amount)), 1)
        threshold = torch.topk(l1_norms, num_keep).values.min()
        masks[name] = (l1_norms >= threshold).cpu().numpy()

    return masks

```

A.6 CONV2D AND BATCHNORM LAYER RECONSTRUCTION

```

def rebuild_conv_layer(conv, out_mask, in_mask=None):
    out_idx = np.where(out_mask)[0]
    new_out = len(out_idx)
    in_idx = np.where(in_mask)[0] if in_mask is not None else None
    new_in = len(in_idx) if in_idx is not None else conv.in_channels

    new_conv = nn.Conv2d(new_in, new_out,
                          conv.kernel_size, conv.stride,
                          conv.padding, conv.dilation,
                          groups=1, bias=(conv.bias is not None))

```

```

w = conv.weight.data
new_conv.weight.data = (w[out_idx][:, in_idx].clone()
                        if in_idx is not None
                        else w[out_idx].clone())
if conv.bias is not None:
    new_conv.bias.data = conv.bias.data[out_idx].clone()
return new_conv

def rebuild_batchnorm_layer(bn, mask):
    idx = np.where(mask)[0]
    new_bn = nn.BatchNorm2d(len(idx))
    new_bn.weight.data = bn.weight.data[idx].clone()
    new_bn.bias.data = bn.bias.data[idx].clone()
    new_bn.running_mean = bn.running_mean[idx].clone()
    new_bn.running_var = bn.running_var[idx].clone()
    return new_bn

```

A.7 MOBILENETV3-SMALL BLOCK RECONSTRUCTION WITH SE RE-SIZE

```

def _rebuild_block_children(container, mask, children_list):
    out_ch = int(mask.sum())
    cnas = [c for c in children_list
            if isinstance(c, (Conv2dNormActivation, nn.Sequential))]
    ses = [c for c in children_list
           if isinstance(c, SqueezeExcitation)]

    # Expansion 1x1 + BN
    exp = cnas[0]
    new_exp_conv = nn.Conv2d(exp[0].in_channels, out_ch, 1, bias=False)
    new_exp_conv.weight.data = exp[0].weight.data[mask].clone()
    new_exp_bn = nn.BatchNorm2d(out_ch)
    for attr in ['weight', 'bias', 'running_mean', 'running_var']:
        getattr(new_exp_bn, attr + ('_data' if 'running' not in attr
                                    else '')).copy_(
            getattr(exp[1], attr)[mask] if 'running' not in attr
            else getattr(exp[1], attr)[mask])
    exp[0], exp[1] = new_exp_conv, new_exp_bn

    # Depthwise + BN
    dw = cnas[1]
    new_dw_conv = nn.Conv2d(out_ch, out_ch, dw[0].kernel_size,
                            dw[0].stride, dw[0].padding,
                            groups=out_ch, bias=False)
    new_dw_conv.weight.data = dw[0].weight.data[mask].clone()
    new_dw_bn = nn.BatchNorm2d(out_ch)

```

```

new_dw_bn.weight.data = dw[1].weight.data[mask].clone()
new_dw_bn.bias.data = dw[1].bias.data[mask].clone()
new_dw_bn.running_mean = dw[1].running_mean[mask].clone()
new_dw_bn.running_var = dw[1].running_var[mask].clone()
dw[0], dw[1] = new_dw_conv, new_dw_bn

# SE block channel resize
if ses:
    se = ses[0]
    se.fc1 = nn.Conv2d(out_ch, se.fc1.out_channels, 1)
    se.fc2 = nn.Conv2d(se.fc2.in_channels, out_ch, 1)

proj = cnas[2]
new_proj = nn.Conv2d(out_ch, proj[0].out_channels, 1, bias=False)
new_proj.weight.data = proj[0].weight.data[:, mask, :, :].clone()
proj[0] = new_proj

```

A.8 EFFICIENTNET BLOCK STRUCTURE ANALYSIS

```

def analyze_block_structure(layer):
    convs = [(n, m) for n, m in layer.named_modules()
              if isinstance(m, nn.Conv2d)]
    if not convs:
        return None, None, None

    dw_idx = next((i for i, (_, m) in enumerate(convs)
                   if m.kernel_size != (1, 1) or
                   m.groups == m.in_channels), -1)
    if dw_idx == -1:
        return None, None, None

    exp_info = convs[dw_idx - 1] if dw_idx > 0 else None
    proj_info = convs[-1]

    if exp_info and exp_info[1].kernel_size != (1, 1):
        exp_info = None # MBConv1 no expansion layer

    return exp_info, convs[dw_idx], proj_info

def extract_efficientnet_masks(model, prune_amount=0.3):
    masks = {}
    for i, layer in enumerate(model.features):
        if i == 0 or i >= len(model.features) - 1:
            continue
        exp, dw, proj = analyze_block_structure(layer)
        if not exp or not dw or not proj:

```

```

        continue
    if exp[1].out_channels != proj[1].in_channels:
        continue

    weight = exp[1].weight.data.abs()
    l1_norm = weight.view(weight.size(0), -1).sum(dim=1)
    keep = max(int(weight.size(0) * (1 - prune_amount)), 1)
    threshold = torch.topk(l1_norm, keep).values.min()
    masks[f"features.{i}"] = (l1_norm >= threshold).cpu().numpy()

return masks

```

A.9 DARTS SEARCH NETWORK AND BI-LEVEL OPTIMISATION

```

class DARTSSearchNetwork(nn.Module):
    def __init__(self, init_channels=16, num_classes=10, layers=6):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, init_channels, 3, padding=1, bias=False),
            nn.BatchNorm2d(init_channels))
        self.layers = nn.ModuleList()
        C = init_channels
        for i in range(layers):
            stride = 2 if i in [layers // 3, 2 * layers // 3] else 1
            C_out = C if stride == 1 else C * 2
            self.layers.append(MixedOp(C, C_out, stride))
            C = C_out

        self.global_pooling = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Linear(C, num_classes)
        self.alphas = nn.ParameterList([
            nn.Parameter(1e-3 * torch.randn(
                len(get_operations(1, 1, 1))))
            for _ in range(layers)
        ])

    def forward(self, x):
        x = self.stem(x)
        for i, layer in enumerate(self.layers):
            x = layer(x, F.softmax(self.alphas[i], dim=0))
        return self.classifier(self.global_pooling(x).flatten(1))

    def genotype(self):
        ops = list(get_operations(1, 1, 1).keys())
        return [ops[a.argmax().item()] for a in self.alphas]

# Bi-level optimisation loop (one epoch, one step shown)
def search_step(search_net, input_w, target_w,

```



```

        input_a, target_a,
        optimizer_w, optimizer_a, criterion):
    # Step A: update weights on training batch
    optimizer_w.zero_grad()
    loss_w = criterion(search_net(input_w), target_w)
    loss_w.backward()
    optimizer_w.step()

    # Step B: update alphas on validation batch
    optimizer_a.zero_grad()
    loss_a = criterion(search_net(input_a), target_a)
    loss_a.backward()
    optimizer_a.step()

```

A.10 ONNX EXPORT AND INT8 QUANTISATION

```

def export_and_quantize_onnx(model, base_filename, onnx_dir):
    model.eval().cpu()
    dummy = torch.randn(1, 3, 32, 32)
    fp32_path = os.path.join(onnx_dir, f"{base_filename}_fp32.onnx")
    prep_path = os.path.join(onnx_dir, f"{base_filename}_prep.onnx")
    int8_path = os.path.join(onnx_dir, f"{base_filename}_int8.onnx")

    # FP32 export
    torch.onnx.export(model, dummy, fp32_path,
                      opset_version=17,
                      do_constant_folding=True,
                      input_names=['input'],
                      output_names=['output'])

    # Graph-optimised Prep export
    try:
        quant_pre_process(input_model_path=fp32_path,
                          output_model_path=prep_path,
                          skip_optimization=False)

        src = prep_path
    except Exception:
        src = fp32_path

    # INT8 dynamic quantisation
    quantize_dynamic(model_input=src,
                    model_output=int8_path,
                    weight_type=QuantType.QUInt8)

    for tag, path in [('FP32', fp32_path),
                     ('Prep', prep_path),
                     ('INT8', int8_path)]:

```

```
mb = os.path.getsize(path) / 1e6
print(f" {tag}: {mb:.2f} MB")
```

A.11 RASPBERRY PI EDGE BENCHMARKING

```
def benchmark_model(onnx_path, n_warmup=10, n_runs=100):
    sess = ort.InferenceSession(
        onnx_path,
        providers=['CPUExecutionProvider'])
    dummy = np.random.randn(1, 3, 32, 32).astype(np.float32)
    feed = {sess.get_inputs()[0].name: dummy}

    for _ in range(n_warmup):
        sess.run(None, feed)

    times = []
    for _ in range(n_runs):
        t0 = time.perf_counter()
        sess.run(None, feed)
        times.append((time.perf_counter() - t0) * 1000)

    avg = statistics.mean(times)
    return {
        'avg_ms' : round(avg, 3),
        'min_ms' : round(min(times), 3),
        'max_ms' : round(max(times), 3),
        'p95_ms' : round(sorted(times)[int(0.95 * n_runs)], 3),
        'std_ms' : round(statistics.stdev(times), 3),
        'fps'    : round(1000 / avg, 2)
    }

def run_benchmark(onnx_dir, n_warmup=10, n_runs=100):
    results = {}
    for fname in sorted(os.listdir(onnx_dir)):
        if not fname.endswith('.onnx'):
            continue

        path = os.path.join(onnx_dir, fname)
        key = fname.replace('.onnx', '')
        results[key] = benchmark_model(path, n_warmup, n_runs)
        results[key]['size_mb'] = round(
            os.path.getsize(path) / 1e6, 2)

    with open(os.path.join(onnx_dir, 'benchmark_results.json'),
              'w') as f:
        json.dump(results, f, indent=2)
    return results
```

A.12 MODEL EVALUATION AND METRICS COLLECTION

```
def collect_metrics(model, testloader, config, label,
                    baseline_inf_time_ms=None):
    model = model.to(config.DEVICE).eval()

    # Accuracy
    correct, total = 0, 0
    with torch.no_grad():
        for imgs, lbls in testloader:
            imgs, lbls = imgs.to(config.DEVICE), \
                           lbls.to(config.DEVICE)
            preds = model(imgs).argmax(dim=1)
            correct += preds.eq(lbls).sum().item()
            total += lbls.size(0)
    accuracy = 100.0 * correct / total

    # Parameter count
    total_params = sum(p.numel() for p in model.parameters())

    # MACs via thop
    dummy = torch.randn(1, 3, 32, 32).to(config.DEVICE)
    macs, _ = profile(model, inputs=(dummy,), verbose=False)

    # Inference latency
    for _ in range(config.INFERENCE_WARMUP_BATCHES):
        with torch.no_grad():
            model(dummy)
    t0 = time.time()
    for _ in range(config.INFERENCE_MEASURE_BATCHES):
        with torch.no_grad():
            model(dummy)
    inf_time_ms = (time.time() - t0) / \
                  config.INFERENCE_MEASURE_BATCHES * 1000

    speedup = (baseline_inf_time_ms / inf_time_ms
               if baseline_inf_time_ms else 1.0)
    latency_reduction = (1 - inf_time_ms / baseline_inf_time_ms
                        if baseline_inf_time_ms else 0.0) * 100
```

Appendix B

Patent Details

- **IPR Number:** R-IPR0002246P
- **Title:** A Computer-Implemented Method for Simultaneous Neural Architecture Search and Structured Channel Pruning
- **Status:** Under Process